Filesystem

Agustín Merino

Julio 2025

1. Concepto (Carpeta Postan y videos)

Un sistema de archivos (File System) sirve para gestionar información persistente. Debe permitir:

- Guardar información
- Consultarla
- Actualizarla
- Eliminarla

El sistema de archivos es, en realidad, una abstracción provista por el SO que le da semántica al disco. El disco tiene todos sus datos como bytes puros. Un sistema de archivos le permite al SO establecer un orden en dicho disco. Sin él, el sistema no sabría cómo interpretar los datos en el medio de acceso que se encuentran.

Típicamente se usan medios magnéticos. Pueden ser cintas, discos magnéticos, de estado sólido, etc.

La información en dichos discos se almacena a través de bloques, los cuales son conjuntos de N bytes, típicamente unos 512 bytes. Todas las transferencias (lecturas, escrituras) se realizan de a bloques, no de a bytes individuales.

Todo sistema de archivos contará con dos cosas centrales:

- La información que tiene que guardar (data).
- La información extra para la gestión (metadata). Esta generalmente consta de fecha de creación, tamaño, nombre, etc..

A continuación se presentan algunos sistemas de archivos.

Aclaración: Se utilizará la palabra *disco* para referirse al medio físico de almacenamiento. Este puede no ser necesariamente un disco en sí, sino una cinta o un medio de estado sólido.

1.1. TAR (UNIX)

Tiene la siguiente estructura



Figura 1: Estructura del sistema de archivos TAR

Tiene como referencia las unidades de cintas. A medida que voy guardando archivos en el disco, debo guardar su cabecera (*header*) la cual contiene la metadata, y posteriormente, sus datos. La cabecera ocupa un bloque.

El relleno * se usa en caso de que el archivo no llegue a ocupar bloques completos. Los bloques no se comparten entre archivos distintos. Como se puede notar, esto ocasiona que siempre haya fragmentación interna.

¿Qué sucede si quiero modificar un archivo? Si hay espacio para mi modificación en el bloque asignado a los datos del archivo, simplemente los escribo. El problema viene cuando no hay espacio suficiente y debo agregar más bloques. A menos que el archivo se encuentre al final del disco, debería marcar el archivo como borrado y reescribirlo al final.

- Ventajas
  - Simple.
- Desventajas
  - Acceso secuencial.
  - Actualización compleja.
  - Si los bloques del medio de almacenamiento tienen son mayores a 512 Bytes, los archivos estarán en un sub-bloque.
  - Para saber si un archivo está presente en el disco, debo, potencialmente, recorrerla en su totalidad, cabecera por cabecera.

Es útil para ocasiones donde las actualizaciones no son necesarias.

1.2. Commodore 64 (concentrar la metadata)

Debido a la última desventaja del sistema TAR, se propuso como idea concentrar toda la metadata, o sea, los *headers*, al comienzo del disco.

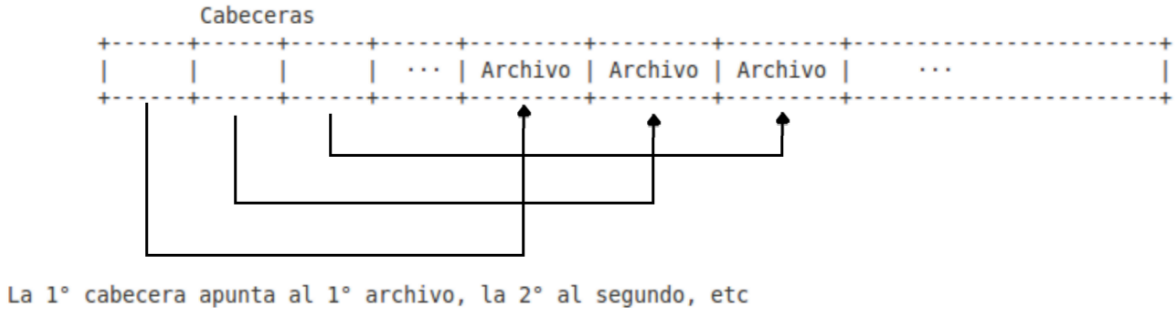


Figura 2: Commmodore 64 File System.

De esta manera, para verificar si un archivo está presente (o cualquier metadato de algún archivo) se obtiene leyendo los primeros bloques del disco.

En este caso, cada cabecera tiene un puntero al bloque donde comienza el archivo.

Sin embargo, tengo el mismo problema para agrandar archivos más allá del tamaño del bloque en el que está contenido ya que los archivos están pegados entre sí.

Además, el tamaño del sector de cabeceras es fijo, por lo que tendré un límite de archivos.

- Ventajas
  - Simple.
  - Es más rápido encontrar los archivos.
- Desventajas
  - Actualización compleja.
  - Tamaño fijo del sector de cabeceras.
  - Si los bloques del medio de almacenamiento tienen son mayores a 512 Bytes, los archivos estarán en un sub-bloque.

1.3. Questar/MFS

En busca de solucionar el problema de tener los archivos completos uno al lado del otro (problema el cual hacía muy difícil la actualización), este sistema de archivos trató de permitir la fragmentación de los mismos.

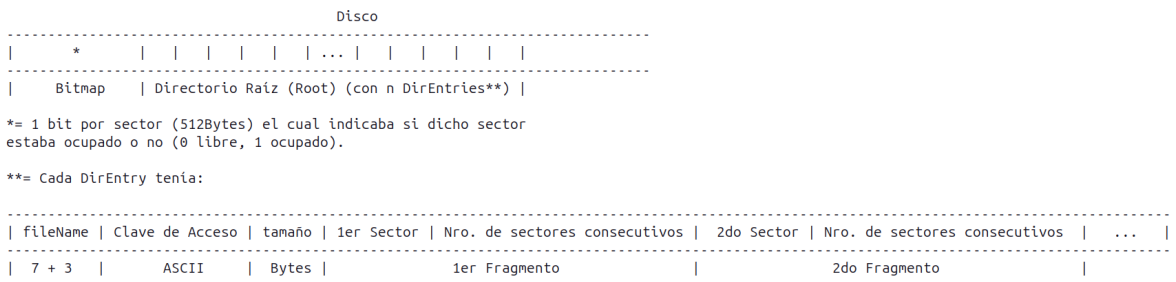


Figura 3: Questar File System.

El mapa de bits (*bitmap*) está presente en el NachOS. Es una estructura que auspicia de lista representativa de los sectores en el disco. En esta estructura se marca qué sectores están ocupados o libres. En NachOS, se encuentra al principio del disco, como acá, en el sector 0.

El catálogo de directorios sería el *directorio raíz*. Cada entrada del directorio raíz tendrá los metadatos del archivo, es decir, las cabeceras.

El nombre podía ser de 7 caracteres máximo. Los 3 que se suman son la extensión (el punto no se guardaba).

La clave de acceso se guardaba en ASCII y daba una forma de obtener un acceso controlado al mismo.

El archivo se divide en fragmentos. Cada fragmento en el header consta del sector de inicio y un número indicando cuántos fragmentos consecutivos a este existen. Es decir, si el 1er sector era el 200 y se tenían 5 sectores consecutivos, ese fragmento era el conjunto de sectores $\langle 200, 201, 202, 203, 204, 205 \rangle$. Luego, el 2do fragmento no es necesario que inicie en el 206, y ahí es donde se permitía la fragmentación.

Cuando se crea un archivo, se ve si se puede agregar en el último fragmento. Para que pueda hacerse, debe haber sectores libres a continuación del primer sector donde comienza el fragmento. De no ser posible, se crea un nuevo fragmento y comienzan a agregarse desde allí.

Una desventaja, al igual que los anteriores, es que existe fragmentación interna en el caso que un archivo no use un sector completo.

Problema: Fragmentación acotada. Una vez que se superan los n fragmentos, ya no puede seguir fragmentándose más el archivo.

NOTA: Un subdirectorio es simplemente un archivo el cual sus fragmentos son DirEntries.

1.4. MSDOS FS

A principio de los 80, Microsoft comienza a utilizar este File System. Se basa en unificar dos aspectos a manejar:

- Gestión del espacio libre.
- Permitir y gestionar la fragmentación arbitraria. Es decir, no tener un número de fragmentos acotados, como se tenía en QuestarFS.

Para lograr esto, se utilizó como estructura principal una estructura denominada FAT (*File Allocation Table*).

Dicha estructura tiene esta forma:

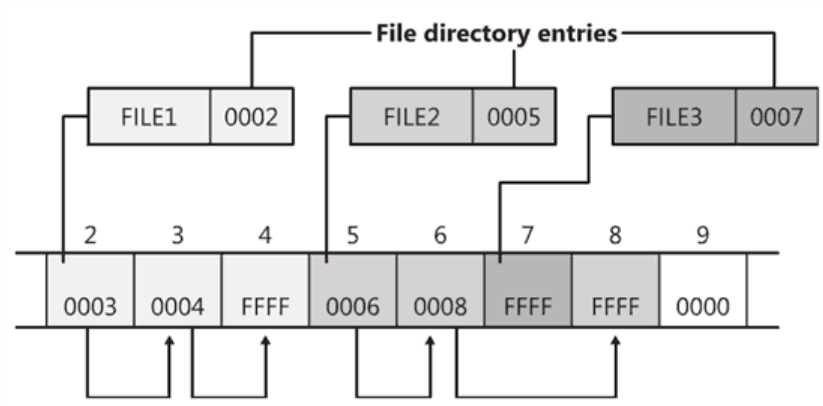


Figura 4: File Allocation Table

Los valores que podían tener los sectores representados en la FAT son punteros de N bytes, donde N depende del tamaño de la FAT (FAT-16, FAT-32, etc..) los cuales pueden tener los siguientes valores:

- **0**: El sector está libre.
- **FFFFFE**: Sector dañado (seteado durante el formateo).
- **FFFFFF**: Último sector que le pertenece a un archivo.
- **Otro**: Sector en uso por un archivo. El número asignado indica el siguiente sector.

La FAT permite representar la información de los sectores de un archivo como una lista enlazada.

Las escrituras en la FAT deben ser **síncronas** para evitar un apartado de problemas relacionados con la inconsistencia del disco. Esto se explica mejor en la sección 1.6.

La estructura general del MSDOS File System, que utilizaba la FAT era:

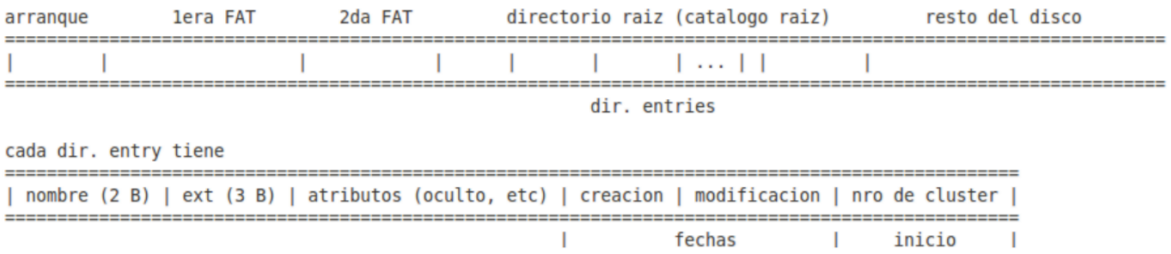


Figura 5: MSDOS File System

Los datos de este File System, no se organizan en sectores, sino más bien en **clusters**, es decir **grupos de sectores**. Cada cluster contiene 4 sectores, y cada sector 512 Bytes. Por lo tanto, **cada cluster es de 2048Bytes (2KB)**.

Por lo tanto, las entradas de la FAT serán clusters, no sectores.

En el cluster de arranque se guardan los datos necesarios para el booteo del sistema (firmware UEFI/BIOS).

Se guardan 2 copias de la FAT para seguridad, si se llega a dañar el bloque donde se encuentra la primera FAT, se perdería toda la estructura de los archivos.

Los metadatos se guardan en un directorio raíz, al igual que en QuestarFS, solo que cada header no contiene información de los fragmentos. En su lugar, posee la dirección del primer cluster que contiene los datos del archivo. A partir de este cluster, el SO puede ir resolviendo la ubicación de los siguientes haciendo uso de la FAT.

- Ventajas
  - Es un buen FS para discos chicos.
- Desventajas
  - La FAT ocupa mucho espacio y hay que mantenerla siempre en memoria principal. Cada entrada de la FAT es un entero (4 bytes) por cada sector del sistema. Si el

disco es de 20GB con un tamaño de sector de 1KB, la tabla necesitará 20 millones de entradas, una para cada uno de los 20 millones de sectores. Como cada entrada almacena un entero de 4 bytes, la FAT terminará pesando unos 80MB. Esto será, tener ocupados 80MB de memoria ram todo el tiempo. Puede ponerse en memoria paginable (podría escribirse en SWAP), pero seguiria ocupando una gran cantidad de memoria virtual y espacio en disco, asi como generar tráfico de paginación extra.

- La política de actualización síncrona de la FAT lentificaba la escritura.

1.5. UNIX FS

Se comenzó a desarrollar a principio de los 70, antes de FAT. Es la base de la mayor parte de los sistemas de archivos basados en los derivados de Unix, Linux, etc..

Su concepto principal es el **i-nodo** (*information node*). La estructura del disco es la siguiente:

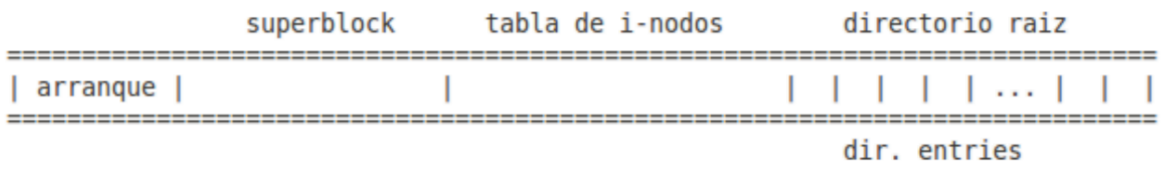


Figura 6: UNIX File System

Al principio, al igual que MSDOS, tendremos el arranque, espacio del disco reservado para los programas cargadores del SO (bootloaders). [1].

Luego, sigue el **superblock**, el cual contiene información sobre todos los parámetros del disco, como está formateado, cantidad de i-nodos, última fecha de montaje, número mágico que identifica el tipo de sistema de archivos, etc.. (información administrativa).

Particularmente, el superblock tiene:

- El bitmap.
- Lista enlazada de i-nodos libres.

La tabla de i-nodos posee información de todos los i-nodos del sistema, es decir, de todos los archivos, ya que necesito un i-nodo por cada archivo.

Finalmente, se encuentra el root (directorio raíz), el cual contiene todas las DirEntries. Sin embargo, en este File System, las DirEntries son mucho más sencillas debido a que la información se guarda ahora en los i-nodos.

Se define ahora el concepto de **i-nodo**.

[1] UEFI, la nueva versión de BIOS, es una especificación que define una interfaz entre el sistema operativo y el firmware de la plataforma. El firmware es el programa embebido en la motherboard encargado de leer y ejecutar los primeros 512Bytes del disco de arranque, que es usualmente donde se encuentra el bootloader

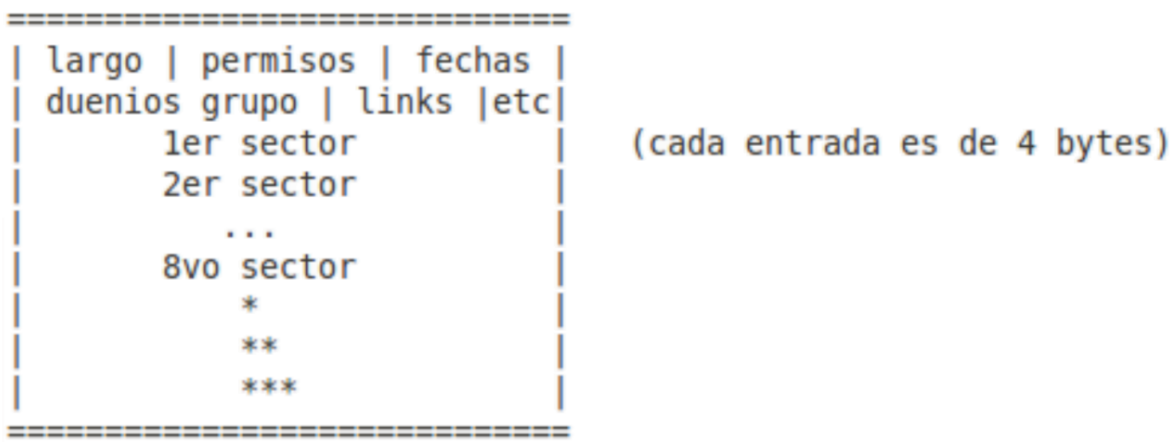


Figura 7: I-Nodo

Un **i-nodo** es una estructura que generalmente posee un tamaño fijo (un bloque - 512Bytes) y posee toda la metadata de un archivo.

Un campo interesante es el de los links. Vemos que ahora las cabeceras no mantienen más el nombre del archivo. Ahora los archivos pueden tener nombres idénticos entre sí. Lo que importa es que el archivo creado haga referencia al i-nodo correspondiente en el disco. El campo links dice cuántas referencias actuales tiene el archivo. Si dicho campo llega a 0, significa que el archivo no será más direccionable y puedo liberarlo (se borró el archivo). Este campo trata sobre los **hard links**, no los **soft links** (concepto explicado más adelante).

El i-nodo contiene una lista de sectores direccionables directamente.

* es un puntero que apunta a una tabla de 128 entradas de 4 bytes (un entero) que indica dónde se encuentran los siguientes sectores (en este caso el 9, 10, 11,...).

** Se conoce como 2 indirecciones. Básicamente apunta a una tabla como * pero, en vez que sus entradas apunten a sectores, apuntan a tablas del tipo *. Es decir, tiene 128^2 sectores. Esta hay que implementar en NachOS.

*** Se conoce como 3 indirecciones y sigue la misma idea.

En este diagrama de un i-nodo, puede verse más detalladamente:

---

del SO. Además, se encarga de funciones como: inicializar hardware, mostrar el setup, buscar el dispositivo de arranque, etc..

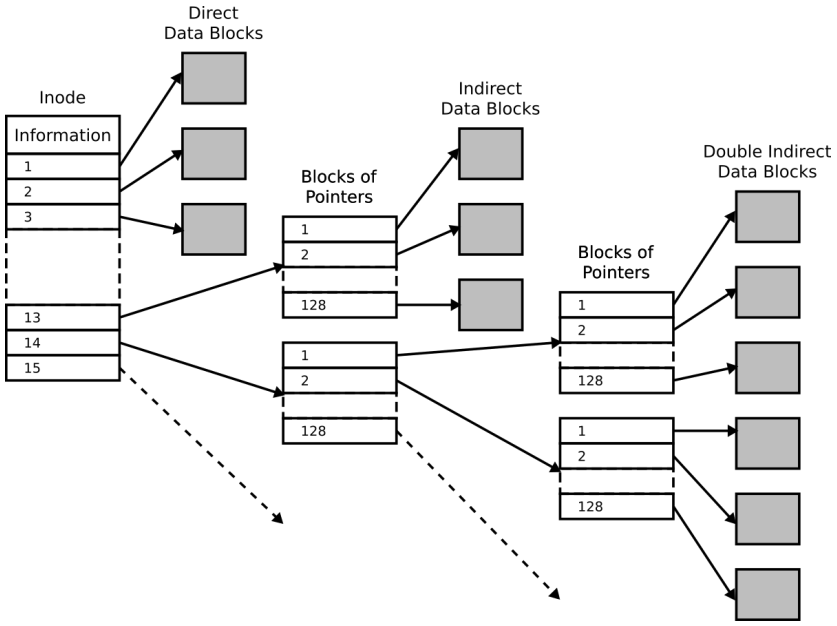


Figura 8: Niveles de indirección del i-nodo en detalle.

En NachOS el **i-nodo** es el **fileHeader**.

Por otro lado, las DirEntries, son de la forma:

```
==============================
| nombre (14 bytes) | i-node |
==============================
```

Figura 9: DirEntry en UnixFS

Vemos que guardan el nombre del archivo (junto con su extensión) y el número de su i-nodo, mediante el cual lo puedo buscar en la tabla de i-nodos (que estará cargada en memoria). Una vez encontrado el i-nodo, ya tengo toda la información necesaria para manipular el archivo.

Vemos que incluso, un archivo puede tener distintos nombres, al contrario que FAT. Pueden haber muchas entradas de directorios distintas que apunten al mismo i-nodo. Por lo tanto, si tengo dos archivos con dos nombres distintos (dos dirEntries distintas) que apuntan al mismo i-nodo, cada vez que modifique algo en uno de ellos, se reflejará en el otro (debido a que se trata del mismo i-nodo).

Vemos entonces que la cuenta de links en el i-nodo representa el número de cabeceras apuntando a dicho i-nodo.

Borrar un archivo ahora es borrar una dirEntry. Cada vez que borro una dirEntry, bajo la cuenta de links del i-nodo. Cuando llega a 0, se puede reclamar todo el espacio del archivo (sectores ocupados) como libres en el bitmap.

Recordar que el i-nodo ocupa un sector, por lo que también tendrá un lugar en el bitmap.

1.6. Mejorando el FS

El sistema de archivos introducido por **BSD** [2], basado en Unix FS introdujo montones de mejoras, principalmente de rendimiento, las cuales hicieron que durante mucho tiempo fuera considerado un sistema de archivos de referencia.

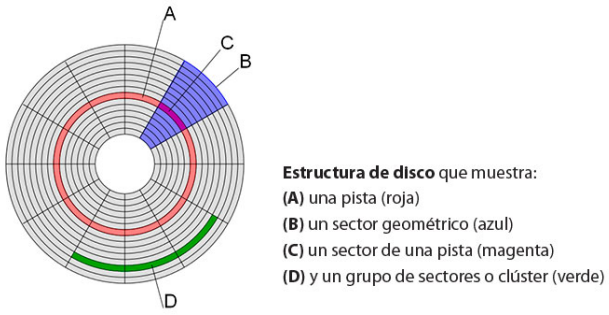
1.6.1. Hacer crecer el sector lógico

BSD introdujo el concepto de **sectores lógicos** distintos de los sectores físicos. Vemos que FAT agrupa lógicamente sectores en **clusters**. Aquí se hace algo muy parecido llamándolos **sectores lógicos**. Un sector lógico en BSD era de 4096 Bytes (4KB) en lugar de 512 Bytes que tiene un sector físico. Aunque los programas lean de a un sector (512 Bytes), el SO pedía un sector lógico (4KB - 8 sectores). De esta forma, cuando el programa pedía el próximo sector, este ya se encontraba en memoria. Esto dió una mejora del 20 % en la velocidad.

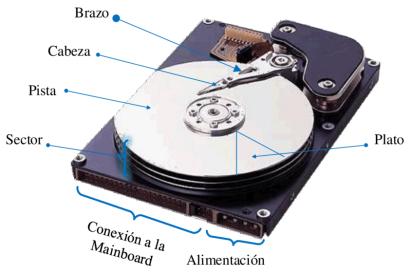
1.6.2. Algoritmo del ascensor

Para explicar esta mejora, primero veamos la forma del medio de almacenamiento físico basado en discos magnéticos, los cuales, por el momento, siguen siendo ampliamente utilizados.

Podemos ver dicho dispositivo en las siguientes figuras:



(a) Estructura de un plato de un disco rígido.



(b) Partes de un disco rígido.



(c) Cabezas de un disco rígido.

Figura 10: Formato de un disco rígido.

Los platos contienen todos los bytes del disco. Giran todos juntos, a la misma velocidad, y las cabezas (especie de púas duales, una para la parte superior del plato y otra para la

[2] BSD empezó como un conjunto de mejoras al Unix original de AT&T, pero con el tiempo se convirtió en un sistema completo por derecho propio. Muchas ideas y componentes que nacieron en BSD terminaron siendo adoptadas por otros sistemas Unix (y también por sistemas no-Unix como Windows y macOS).

inferior) van leyendo todas al mismo tiempo, en el mismo lugar en los distintos discos.

Manteniendo las cabezas fijas, se leen todos los datos de una misma **pista**. El conjunto de pistas de cada plato forman un **cilindro**. Dentro de una **pista**, se puede grabar/leer información dentro de *sectores*. Es decir, una **pista** está formada por un **conjunto de sectores**.

A su vez, las cabezas poseen un pequeño motor que permite modificar su ángulo, permitiendo desplazarse entre las pistas.

Por lo tanto, para referenciar a los bloques de un disco rígido se usa un método **CHS** (*Cilinder, Header, Sector*).

Notar que los sectores son todos del mismo tamaño y el diámetro de las pistas va variando. Esto significa que las pistas más cercanas al centro tendrán los datos más comprimidos dentro de los sectores, mientras que en las más alejadas del centro, los sectores presentarán sus datos más dispersos. Actualmente, se han agregado 'métodos' que no requieren dispersión en los datos de las pistas con mayor diámetro aprovechando así mucho más el espacio.

En resumen, leer todos los datos que están dentro de una pista, es decir, todos los sectores, es muy rápido (si un disco gira a 7200 rpm, en 1/7200 de minuto - algo así de 8.33 milisegundos - puedo leer una pista completa). Sin embargo, mover el cabezal (se mueve con un motor paso a paso) es muy lento en comparación.

Cabe aclarar que esta forma de organizar los datos en el disco ya quedó **obsoleta**. Existen nuevos métodos y dispositivos mucho más rápidos.

¿Qué propuso BSD en base a esto?

Comencemos con un ejemplo, supongamos que tenemos que acceder (independientemente de lectura o escritura) a las siguientes pistas (se debería usar mecanismo CHS pero nos interesa evaluar el rendimiento):

$$\langle 1, 10, 5, 2, 9, 3 \rangle$$

Creemos la secuencia de pistas que tiene que recorrer para pasar de la pista i a la pista $i + 1$, asumiendo que comenzamos en la pista 1:

$$\langle 9, 5, 3, 7, 6, 9 \rangle$$

Es decir, de la 1 a la 10, hay 9 pistas de distancia, de la 10 a la 5, hay 5, etc..

Sumando esta secuencia, vemos que la cantidad de pistas que tenemos que saltar para hacer todos los pedidos, es:

$$9 + 5 + 3 + 7 + 6 + 9 = 39$$

Esto quiere decir, que en el orden de pistas propuesto, deberemos recorrer 39 pistas para poder realizar los pedidos.

La mejora de BSD fué, **ir escribiendo en caché todos los accesos a las pistas, guardarlos en memoria y ordenarlos para evitar largos recorridos**.

Es decir, los accesos ahora serán:

$$\langle 1, 2, 3, 5, 9, 10, 12 \rangle$$

y la secuencia de pistas a recorrer ahora es:

$$\langle 1, 1, 2, 4, 1, 2 \rangle$$

Es decir, un total de:

$$1 + 1 + 2 + 4 + 1 + 2 = 11$$

Terminaremos recorriendo en total 11 pistas.

A este algoritmo se le conoce como **algoritmo del ascensor** debido a su modo de funcionamiento al ir recorriendo los pisos en orden (si aprieto los pisos 1,10,3,7 comenzando del 0, cualquier ascensor mínimamente funcional recorrerá en orden: 1,3,7,10).

Sin embargo, esto no siempre es óptimo. **El algoritmo del ascensor nos garantiza su optimicidad en tanto los pedidos de lectura/escritura sean independientes**.

Por ejemplo, imaginemos que estamos borrando un archivo utilizando UnixFS.

Los pasos a seguir son:

1. Deberé buscar el archivo. Para eso deberé recorrer el directorio correspondiente buscando el nombre del archivo para así obtener su i-nodo.

2. Deberé modificar el directorio para marcar la entrada de directorio correspondiente al archivo borrado como libre.

3. Finalmente, debo ir al i-nodo, y si la cuenta de links pasa a 0, debo liberar el i-nodo (marcarlo como libre en la tabla de i-nodos) y marcar como libres en el bitmap todos los sectores presentes en dicho i-nodo.

Pero, ¿qué sucede en caso de falla entre el paso 2 y 3? **Existirá un i-nodo que no está referenciado desde ningún directorio** y, por lo tanto, **jamás podré reclamar los sectores correspondientes al archivo y habré perdido espacio**.

Esto, sin embargo, no es demasiado problema ya que existen métodos que permiten checkear el disco y buscar i-nodos **huérfanos** (i-nodos sin nombre que los referencie). A estos i-nodos los puedo intentar **recuperar**, lo cual, en caso qué el sistema no pueda reconstruir

su estructura general los guardará en un directorio especial llamado `lost+found` (se explica mejor más adelante), o simplemente el sistema puede reclamar el espacio asumiendo que pertenece a un archivo eliminado.

Pero ¿Qué sucede si hubiera hecho las cosas al revés y el sistema falla? Es decir, invirtiendo 2 y 3 y el sistema fallando al terminar la nueva 2. En este caso, habré marcado el i-nodo como libre pero no liberé su entrada de directorio. Es decir, en mi directorio todavía existe un archivo que apunta al i-nodo libre. Si se crea un nuevo archivo, y se le da dicho i-nodo, tendré dos archivos no relacionados pero apuntando al mismo i-nodo y, por lo tanto, todos los cambios que haga sobre uno se reflejarán en el otro.

En este caso, el sistema de archivos queda mucho más corrupto.

Entonces, en operaciones como creación o eliminación de archivos, es importante que los metadatos se escriban en un determinado orden.

Otra cosa que se propuso fue el llamado **interleaving**. Esto sí que es obsoleto hoy en día, pero a grandes rasgos la idea es que: los discos giran tan rápido que si los sectores están contiguos puedo pasarme de alguno que necesitaba leer y tener que dar toda la vuelta. Es por esto que puedo dejar un espacio entre sectores contiguos para equiparar los tiempos de lectura/escritura y revoluciones del plato.

1.6.3. Distribución del superblock, y demás estructuras

Muy probablemente, a medida que use el disco, estando en cualquier parte del mismo, necesite información del superblock o de la tabla de i-nodos. Recordemos que estos se encuentran al principio del disco, en los primeros sectores. Es por esto que BFS optó por distribuir estas estructuras a lo largo de todo el disco. Sin embargo, no se tratan de copias exactas de dichas estructuras, sino que existe una noción de *localidad*. Es decir, se tiene el superblock, bitmap y tabla de i-nodos específicos a cada grupo de cilindros cerca de estos. De esta forma, se obtiene una ganancia en velocidad ya que la información del disco siempre la está relativamente cerca del lugar de trabajo.

Esta distribución no se hace a múltiplos exactos de forma que si se rompe alguna pista/-cabeza no se encuentren todos alineados y se arruinen todas las copias.

Notar que hacer esto también provee mayor seguridad. Tener distintas copias de estas estructuras primordiales para nuestro FS previene que la pérdida de alguna de estas sea algo irreversible.

Estos cambios dieron una mejora de 60 % en velocidad.

Al sistema de archivos de BSD se le llamó **FFS** (*Fast File System*).

1.7. Más mejoras

1.7.1. Eliminando secuencias dependientes

Vimos que el algoritmo del ascensor se rompía en caso qué las operaciones sean dependientes entre sí. Podemos intentar quitar dichas dependencias para aprovechar al completo el algoritmo del ascensor.

Esto fue lo que introdujeron los FS *ext* y *ext2*. Logró ignorar las secuencias dependientes estableciendo mecanismos o programas que revisan el disco completo buscando estructuras inconsistentes en el FS y solucionar las inconsistencias. Por ejemplo, el programa `fsck` (*file system check*) de los *ext* es mucho más complicado.

1.7.2. Cambiando la estructura del directorio

Tomemos la apertura de un archivo. El kernel debe:

- Buscar en el directorio correspondiente (un directorio es un array de dirEntries $\rightarrow O(n)$).
- Sacar su i-nodo de información.
- Checkear los accesos.
- Construir el descriptor en el System Segment.

Podemos acelerar esto cambiando el array de dirEntries por otra estructura. Por ejemplo, un árbol. Si está balanceado el costo de búsqueda ahora será $O(log\ n)$. Algunos casos de estos árboles son *Red-Black Tree*, *AVL* y *BTree*.

Los *BTree* (*Balanced Trees*) fueron elaborados en 1971 bajo la premisa de: 'En vez de hacer crecer un árbol desde la raíz, lo hacemos crecer desde las hojas'.

Podemos utilizar dos BTree, uno para los directorios y otro para sectores libres (ordenados por tamaño y ubicación).

El pionero en usar BTrees fue IRIX (el Unix de SGI). Tener dos BTree para los sectores libres es complicado (deben ser consistentes entre sí) y eso lo llevó a tener que incluir **journaling** (*bitácora*).

La idea (se explica más adelante) es que antes de realizar algún cambio, lo escribo en la bitácora. La misma contiene todas las acciones que voy a hacer, pero todavía no hice y se guarda en un sector del disco. Una vez que realizo los cambios, marco en la bitácora que todo anduvo bien. Por lo tanto, ante una eventual falla del sistema, cuando se reanuda su funcionamiento, se mira la bitácora para ver qué cambios no marqué como hechos y poder hacer algo al respecto (realizarlos, por ejemplo).

El journaling, al básicamente ir escribiendo dos veces cada cambio, baja la performance del FS, a costa de aumentar la seguridad.

Es por eso que muchos sistemas de archivos usan journaling únicamente para los **metadatos**. En lugar de registrar los cambios que hace cada programa en particular sobre sus archivos, registro únicamente los cambios en los metadatos de los archivos los cuales son los más importantes al momento de mantener el FS en un estado consistente.

FS con journaling: IRIX(xfs), IBM(jfs), ext3 (ext2 de Linux modificado para journaling), ext4, ResierFS (FS basado en BTree que implementaba cosas como compartir una página con varios archivos), etc..

¿Podemos seguir mejorando esto?

Vimos entonces que cambiando la estructura del directorio de un array a un árbol, mejorábamos la performance. Podemos mejorarlo aún más utilizando una **tabla hash**.

- Array $\rightarrow O(n)$
- BTree $\rightarrow O(log\ n)$
- Hash $\rightarrow O(1)$

Ahora podemos tener una tabla hash de todos los archivos del directorio y buscarlos en $O(1)$. Sin embargo, no es tan práctico ya que cuando actualizo el directorio, debería actualizar la tabla hash.

Por otro lado, la **shell** de Linux usa este método para mantener un listado de los ejecutables. Cuando se pone un comando en la línea de comandos, no hay que buscarlo por todo el path, sino que se busca en la hash y puede hacerse en $O(1)$.

Por lo tanto, tener una tabla hash sirve para el caso que las ubicaciones de mis archivos no varíen demasiado (de hacerlo, se debería cambiar la hash recurrentemente).

Este esquema fue usado por BeFS de BeOS usado por BeBox.

¿Podemos seguir mejorando el sistema?

Esto entra ya en el campo experimental. Lo que buscamos entonces es:

- Máxima velocidad
- Máxima seguridad

El problema es que estos dos puntos son antagónicos. Generalmente, lo que ganamos de uno, perdemos del otro. En el caso del journaling por ejemplo, ganamos en seguridad escribiendo los cambios previamente en la bitácora, pero perdemos velocidad al tener que escribir todo dos veces.

Podríamos, por otro lado, hacer que nuestro FS sea, al mismo tiempo, la bitácora. Este caso se llama *log-structured File System* (LogFS) y fue presentado por primera vez en una tesis de doctorado de Margo Seltzer en Berkeley.

La idea de esto es, dividir el disco en segmentos e ir escribiendo los datos unos detrás del otro. Vemos que los sistemas de archivos anteriores, ubicaban los datos de tal forma que

estén cerca de las diversas instancias de estructuras importantes tales como el superblock o el bitmap para así acelerar los tiempos de escritura de los mismos en el disco. LogFS rompe con esto debido a su hipótesis que dice que a medida que aumentan los tamaños de las memorias principales (RAM) en las computadoras modernas, las operaciones I/O serán únicamente pesadas en la escritura, ya que la lectura podrá tenerse casi siempre en caché (en memoria principal). Por lo tanto, LogFS plantea que el disco entero sea una especie de búffer circular que, a su vez, auspicie de log (bitácora) e ir escribiendo los cambios siempre en la cabeza de este búffer.

Pero, ¿Qué sucede con los i-nodos, bitmap, tabla de i-nodos, etc..? La idea es que, al terminar una escritura, pegado a donde se terminan los datos que escribí, se guardan todas estas estructuras ahora actualizadas. Es decir, cada operación deberá escribir nuevamente dichas estructuras para actualizarlas y agregarlas al final. Esto se conoce como ***copy-on-write***, en vez de modificar un bloque, lo copio en otro lado.

Al ser ahora todo secuencial, se mejoran enormemente las lecturas. ¿Pero y las escrituras, quedan penalizadas? Vemos que ahora, los datos, terminan quedando relativamente cerca de los metadatos, lo cual hace que las escrituras no se penalicen demasiado.

Sin embargo, acá vemos un problema... Jamás borramos, va agregando datos. Es decir, tendremos información vieja de la tabla de i-nodos, bitmap, superblock, etc... (osea basura). Este es un inconveniente de este sistema de archivos. Periódicamente requiere el uso de un *garbage collector* que recorra el disco y pueda ir eliminando dicha información ahora inútil en los segmentos utilizados (que no sea el último, que sería el actual).

1.8. Algunos sistemas archivos actuales

- **ext4**: Es una mejora a *ext3* pero ya se dice estar quedando obsoleto (son parches sobre parches ya que el *ext3* es una mejora sobre *ext2* y ext2 es una mejora sobre *extFS*). Las principales mejoras con respecto a *ext3*, son:
  - Soporte de volúmenes de hasta 1024 PiB (discos más grandes).
  - Soporta añadido de *extend*.
  - Menor uso del CPU.
  - Mejoras en la velocidad de lectura y escritura.

  Otra cosa que agregan los nuevos sistemas de archivos es el concepto de ***snapshots*** (se explica más adelante). La idea básicamente es tomar una 'foto' del estado del sistema en un momento particular y guardar la información necesaria para que los bloques mostrados no se liberen y que los bloques nuevos se escriban en otro lado.

- **Btrfs** (*B-tree FS*): Este FS está llamado a ser el sucesor del sistema *ext4*. Está basado en árboles binarios y tiene soporte para cosas tales como *snapshots*, *copy-on-write*, además de presentar mejoras y optimizaciones para discos *SSD*.

Este FS ya no sería un parche sobre parche como vinieron siendo los *ext*, si no que un FS completamente nuevo.

2. Comando útiles de Linux con respecto al FS

Comando	Descripción
`stats`	No es un comando estándar. A veces es un alias o script personalizado para mostrar estadísticas del sistema.
`df -i`	Muestra el uso de inodos por sistema de archivos. Útil para saber si se agotaron los inodos antes que el espacio.
`mkfs.ext2`	Formatea una partición con el sistema de archivos EXT2 (sin journaling).
`mkfs.ext3`	Formatea una partición con EXT3, versión de EXT2 con journaling.
`mkfs.ext4`	Formatea con EXT4, sistema moderno con journaling, extents y mejor rendimiento.
`mkfs.btrfs`	Crea un sistema de archivos Btrfs, que permite snapshots, compresión y otras características avanzadas.
`dumpe2fs`	Muestra información detallada de un sistema de archivos EXT2/3/4.
`debugfs`	Herramienta para examinar y modificar sistemas de archivos EXT2/3/4 a bajo nivel.

Cuadro 1: Descripción de comandos relacionados con sistemas de archivos en Linux

3. Introducción (Cap. 37)

La información *cruda* tiene que pasar una serie de transformaciones.

Yendo de niveles superiores a los más bajos, un programa estructura sus datos en *archivos*, siguiendo el *formato* que resulte más pertinente al tipo de información a presentar. Un conjunto de archivos hoy en día es típicamente representado en una estructura de *directorios*.

Cada dispositivo empleado para almacenar los archivos tiene un directorio.

Los archivos son una estructura meramente lógica; deben ser convertidos para ser representados en un *dispositivo de bloques* como los diversos tipos de unidades como *discos*.

De esta figura podemos ver que el trabajo del sistema operativo es recibir del espacio de usuario las llamadas al sistema que presentan la interfaz de archivos y directorios, integrar el sistema de archivos virtual y traducir la información resultante a un sistema de archivos. Es decir, el SO será el encargado de conectar al usuario con los dispositivos de almacenamiento, ofreciendo una capa de abstracción adecuada y conveniente (archivos, directorios).

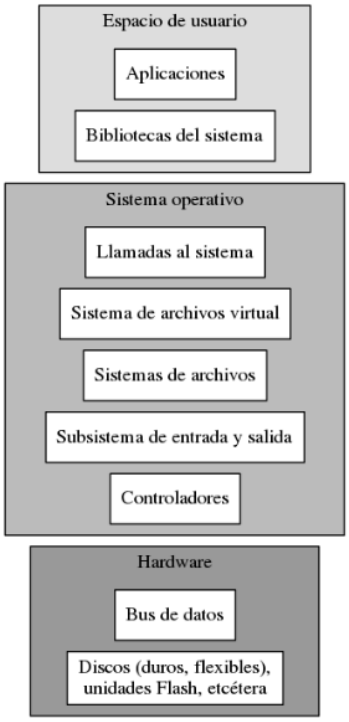


Figura 11: Capas de abstracción para implementar los sistemas de archivos.

4. Concepto de archivo (Cap. 38)

Un archivo es un *tipo de datos abstracto (TAD)*.

Un *TAD*, recordemos, se puede definir como un modelo matemático compuesto por una serie de operaciones definidas sobre un conjunto de datos. Un *TAD* se utiliza para definir un tipo de dato específico en función de las operaciones que se pueden realizar sobre él, sin especificar cómo se implementan (recordar los TADs pilas, colas, etc...).

Decimos que un archivo es un TAD ya que el mismo podría verse como una estructura que exclusivamente permite la manipulación por medio de una interfaz *orientada a objetos*: los procesos en el sistema sólo pueden tener acceso a los archivos por medio de la interfaz ofrecida por el sistema operativo.

En realidad, nuestros datos están almacenados como una secuencia de bytes puros. No poseen delimitaciones acerca de dónde empiezan o terminan, dónde fué escrito por última vez o qué clase de permisos tiene. El concepto de archivo es una abstracción presentada por el sistema al usuario la cual soporta operaciones tales como: read, write, seek, etc...

Para el usuario, los archivos son la *unidad lógica mínima* al hablar de almacenamiento: todo el almacenamiento *persistente* en el sistema al que tiene acceso, se efectúa dentro de archivos; el espacio libre en los diferentes dispositivos no tiene mayor presencia fuera de saber que está *potencialmente* disponible.

Dentro de cada *volumen* (cada medio de almacenamiento), los archivos disponibles con-

forman un *directorio*, y son típicamente identificados por un *nombre* o *ruta*.

4.1. Operaciones con archivos

Cada SO definirá la interfaz de archivos acorde con su semántica, pero en líneas generales, las operaciones que siempre estarán disponibles con un archivo son:

- **Borrar**: Elimina al archivo del directorio y, de ser procedente, libera el espacio del dispositivo.

- **Abrir**: Solicita al SO verificar si el archivo existe o puede ser creado (dependiendo del modo requerido) y se cuenta con el acceso para el *modo de acceso* al archivo indicado y si el medio lo soporta (por ej., a pesar de contar con todos los permisos necesarios, el SO no debe permitir abrir para escritura un archivo en un CD-ROM u otro medio de solo lectura). En C, usamos `fopen()`.

  Al abrir un archivo, el SO asigna un *file descriptor* que identifica la relación entre el proceso y el archivo en cuestión.

  Todas las operaciones descriptas a continuación operan sobre el file descriptor, no con su nombre o ruta.

- **Cerrar**: Indica al sistema que el *proceso en cuestión* terminó de trabajar con el archivo; el sistema entonces debe escribir los buffers a disco y eliminar la entrada que representa a esta combinación archivo-proceso de las tablas activas (file descriptor table), invalidando el *file descriptor*. En C, usamos `fclose()`.

- **Leer**: Si se solicita al sistema la lectura de un archivo hacia determinado buffer, éste copia el siguiente *pedazo* de información. este *pedazo* podría ser una línea o un bloque de longitud definida, dependiendo del modo en que se solicite la lectura. El sistema mantiene un apuntador a la última posición leída, para poder *continuar* con la lectura de forma secuencial. En C, usamos `fread()` la cual entrega el *número de caracteres* especificado; para trabajar con líneas de texto, se podría usar `readline()`.

- **Escribir**: Teniendo un archivo abierto, guarda información en él. Puede ser que escriba desde su primer posición (*truncando* al archivo, esto es, borrando toda la información que ya pudiera tener), o *agregando* al archivo, esto es, iniciando con el apuntador de escritura al final del mismo. En C, usamod `fwrite()`.

- **Reposicionar**: Tanto la lectura como la escritura se hacen siguiendo un *apuntador*, que indica cuál fué la última posición del archivo a la que accedió el proceso actual. Al reposicionar el apuntador, se puede *saltar* a otro punto del archivo. En C, usamos `fseek()`.

Otras funciones más complejas pueden ser combinaciones de estas elementales. *Copy* puede ser una combinación de Crear, Abrir, Leer y Escribir.

Esta semántica para el manejo de archivos presenta a cada archivo como si fuera una *unidad de cinta*, dentro de la cual la cabeza lectora/escritora simulada puede avanzar o retroceder.

4.2. Tablas de archivos abiertos

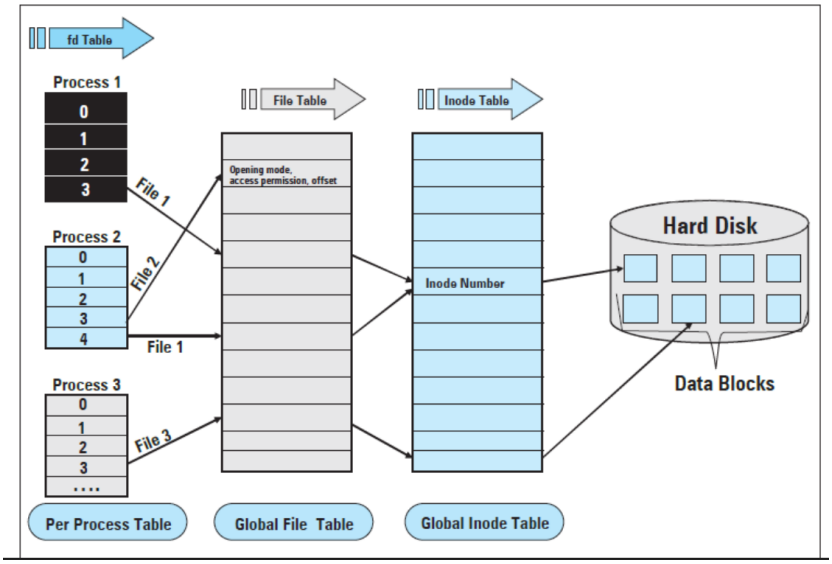


Figura 12: File tables de procesos y SO

Tanto el SO como cada uno de los procesos mantienen normalmente *tablas de archivos abiertos*. Éstas mantienen información acerca de todos los archivos actualmente abiertos, presentándolos hacia al proceso por medio de un *file descriptor*, una vez que el archivo fué abierto, las operaciones que se realizan dentro de éste no son empleando su nombre, si no su *file descriptor*. Cabe destacar que la file table del SO es distinta que la del proceso.

La file descriptor table del proceso únicamente mantiene los archivos que fueron abiertos por dicho proceso y los file descriptor para ese proceso (los file descriptor no son globales para todos los procesos, exceptuando algunos como `stdin`, `stdout` o `stderr` los cuales son 0, 1 y 2 respectivamente).

La información que el SO necesita mantener en su **File Table** es:

- **Conteo de usuarios del archivo**: Cuántos procesos están empleando en todo momento a determinado archivo. Esto permite que, por ejemplo, cuando el usuario solicite *desmontar* una partición o eliminar un archivo, el sistema debe poder determinar cuándo es el momento de declarar la solicitud como *efectuada*. Si algún proceso tiene abierto un archivo y, particularmente, tiene cambios pendientes de guardar, el sistema debe hacer lo posible para evitar que el archivo *desaparezca* de su visión.

- **Modos de acceso**: Aunque un usuario tenga permisos de acceso a determinado recurso, el sistema puede determinar negarlo si llevaría a una inconsistencia.
- **Ubicación en disco**: El sistema mantiene esta información para evitar que cada proceso tenga que consultar las tablas en disco para encontrar al archivo, o cada uno de sus fragmentos.
- **Información de bloqueo**: En caso que los modos de acceso del archivo requieran prohibición mutua, puede hacerlo por medio de un bloqueo.

Por otro lado, el proceso necesita tener en su **File Descriptor Table**:

- **File descriptor**: Relación entre el nombre del archivo abierto y el identificador numérico que maneja internamente el proceso. Un archivo abierto por varios procesos puede tener descriptores de archivos distintos en cada uno de ellos. El file descriptor de archivo otorgado por el sistema a un proceso es simplemente un número entero, que podría entenderse como el *n-ésimo archivo empleado por el proceso*.
- **Permisos**: Los modos válidos de acceso para un archivo. Esto no necesariamente es igual a los permisos que tiene el archivo en cuestión en disco, sino que es el *subconjunto* de dichos permisos bajo los cuales está operando para este proceso en particular.

En un SO multitareas (básicamente todos en este siglo), más de un proceso podría abrir el mismo archivo a la vez; lo que cada uno de ellos pueda hacer, y cómo esto repercute en lo que vean los demás procesos, depende de la semántica que implemente el sistema.

4.3. Acceso concurrente: bloqueo de archivos

Los archivos pueden emplearse como un mecanismo de comunicación entre procesos que no guarden relación entre sí (IPC). Además, para emplear un archivo basta con indicar su nombre o ruta. Es por esto que los SO multitarea implementan mecanismos de bloqueo para evitar que varios procesos intentando emplear de forma concurrente a un archivo se corrompan mutuamente.

Algunos SO permiten establecer bloqueos sobre determinadas regiones de los archivos, aunque la semántica más común es operar sobre el archivo entero.

En general, la nomenclatura que se sigue para los bloqueos es:

- **Compartido** (*Shared lock*): Equivalente a un bloqueo para lectura - varios procesos pueden adquirir al mismo tiempo un bloqueo de lectura, e indica que todos los que posean dicho *lock* tienen la expectativa de que el archivo no sufrirá modificaciones.
- **Exclusivo** (*Exclusive lock*): Un *lock* exclusivo puede ser adquirido por un sólo proceso, e indica que realizará operaciones que modifiquen el archivo.

Recordar el problema de los lectores y escritores.

Respecto al *mecanismo* de bloqueo hay también dos tipos, dependiendo qué tan explícito tiene que ser su manejo.

- **Mandatorio y obligatorio** (*Mandatory locking*): Una vez que un proceso adquiere un lock obligatorio, el SO se encargará de imponer las restricciones correspondientes de acceso a todos los demás procesos, independientemente de si estos fueron programados para considerar la existencia de dicho bloqueo o no. **El SO se encarga**.

- **Consultivo o asesor** (*Advisory locking*): Manejado cooperativamente entre los procesos involucrados, y depende del programador de *cada uno* de los programas en cuestión el solicitar y respetar dicho bloqueo. **Cada proceso se encarga**.

Los mecanismos que emplean mutexes, semáforos o variables de condición serían *consultivos*.

Únicamente los que emplean monitores (en que la única manera de llegar a la información es por medio del mecanismo que la protege) serían *mandatorios*.

Como regla general, en los sistemas **Windows** se maneja un esquema de bloqueo **obligatorio**, y en sistemas **Unix** es de bloqueo **consultivo** [3].

El manejo de bloqueo por archivos requiere del mismo cuidado que el de bloqueo por recursos. Dos procesos intentando adquirir un lock exclusivo sobre dos archivos pueden caer en un bloqueo mutuo (deadlock).

4.4. Tipos de archivo

Existen archivos de diferentes tipos: documento de texto, binario ejecutable, archivo de audio o video, etc... Intentar emplear un archivo como uno de un tipo distinto puede resultar desde una frustración al usuario porque el programa no responde como éste quiere, hasta en pérdidas económicas o infecciones.

Hay tres estrategias principales para que el SO reconozca el tipo de un archivo:

- **Extensión**: En los sistemas CP/M de los setenta[4], el nombre de cada archivo se dividía en dos porciones, empleando como elemento separador al punto. Las dos porciones eran

[3] Por eso es tan común que en Windows el sistema mismo rechace hacer determinada operación porque el *archivo está abierto por otro programa* (bloqueo mandatorio compartido), mientras que en Unix esta responsabilidad recae en cada uno de los programas de aplicación.

[4] CP/M, siglas de «Control Program for Microcomputers», fue un sistema operativo pionero para microcomputadoras desarrollado en la década de 1970 por Digital Research. Fue el primer sistema operativo popular para computadoras personales, conocido por su independencia del hardware. CP/M permitía a los fabricantes de computadoras que utilizaban el procesador Intel 8080 ejecutar aplicaciones de terceros, lo que contribuyó a su adopción generalizada.

el nombre del archivo y su extensión. El sistema mantenía una lista de extensiones conocidas, para las cuales sabría cómo actuar, y este diseño se propagaría a las aplicaciones, las cuales sólo abrirían aquellos archivos cuyas extensiones supieran manejar.

Esta estrategia fué heredada por VMS y MS-DOS[5], de donde la adoptó **Windows**; ya en el contexto de un entorno gráfico, Windows agrega la relación de extensiones con los programas capaces de trabajar con ellas, permitiendo invocar un programa con sólo dar "doble click.[en] un archivo.

Este esquema de asociación de tipo de archivo permite ocultar las extensiones toda vez que ya no requieren ser del conocimiento del usuario, si no únicamente por el SO. Esto abre una vía de ataque automatizado que se popularizó en su momento: el envío de correos con extensiones engañosas duplicadas. Esto es, un programa maligno (*troyano*) se envía a todos los contactos del usuario infectado, presentándose por ejemplo como una imagen, con el nombre `inocente.png.exe`. Por el esquema de ocultamiento mencionado, éste se presenta al usuario como `inocente.png`, pero al abrirlo, el SO lo reconoce como un ejecutable, y lo ejecuta en vez de abrirlo en un visor de imágenes.

- **Números mágicos** (*magic numbers*): Empleada por los sistemas **Unix**. El sistema mantiene una lista compilada de las *huellas digitales* de los principales formatos que debe manejar, para reconocer el contenido de un archivo basado en sus primeros bytes [6].

  Casi todos los formatos de archivo incluyen lo necesario para que se lleve a cabo este reconocimiento y, cuando no es posible hacerlo, se intenta por medio de ciertas reglas *heurísticas*. Por ejemplo, todos los archivos de imagen en *formato de intercambio gráfico (GIF)* inician con la cadena `GIF87a` o `GIF89a`, dependiendo de la versión, los *Formato de Documentos Portátiles (PDF)* con *%PDF*. Algunos de estos formatos no están *anclados* al inicio, sino en un punto específico del primer bloque.

  Un caso especial de números mágicos es el llamado `hashbang (#!)`. Esto indica a un sistema Unix que el archivo en cuestión (típicamente un archivo de texto, incluyendo código fuente en algún lenguaje de *script*) debe tratarse como un ejecutable, y empleando como *intérprete* al comando indicado inmediatamente después del *hashbang*. Es por esto que se pueden ejecutar directamente, por ejemplo, los archivos que inician con `#!/usr/bin/bash`: el SO invoca al programa `/usr/bin/bash`, y le especifica como

[5] VMS (Virtual Memory System) y MS-DOS son sistemas operativos, pero con características muy diferentes. VMS, desarrollado por DEC, es un sistema operativo robusto y potente, diseñado para entornos multiusuario y multitarea en minicomputadoras, con énfasis en confiabilidad y gestión de recursos. MS-DOS, por otro lado, es un sistema operativo de un solo usuario y tarea, conocido por su simplicidad y por ser el sistema operativo dominante en las PCs de la década de 1980.

[6] Una de las ventajas de esqe esquema es que cada administrador de sistema puede ampliar la lista con las huellas digitales que requiera localmente.

argumento al archivo en cuestión.

Comando UNIX: `chmod +x script.sh` hace al archivo `script.sh` ejecutable y permite ejecutarlo haciendo `./script.sh`. Hay que especificar el directorio actual (.) ya que el mismo no está en el *PATH* por defecto.

El *PATH* es una variable de entorno **Unix** que contiene una lista de directorios donde el sistema busca ejecutables cuando se intenta ejecutar un comando.

Básicamente, cuando se escribe un comando en la terminal, el sistema revisa la lista de directorios definida en PATH para localizar el archivo ejecutable correspondiente al comando que se ha introducido.

Es una cadena de texto que contiene una lista de directorios separados por dos puntos («:»). Cuando se ejecuta un comando, el sistema busca el ejecutable en cada directorio de la lista, en el orden en que aparecen, hasta que lo encuentra o llega al final de la lista.

Por ejemplo, si el PATH contiene `/usr/bin:/bin:/usr/local/bin`, el sistema buscará primero en `/usr/bin`, luego en `/bin` y finalmente en `/usr/local/bin`. Si el ejecutable se encuentra en `/bin/ls`, por ejemplo, el sistema lo ejecutará desde allí.

- **Metadatos externos**: Los sistemas de archivos empleado por las **Apple Macintosh** desde 1984 separan en dos *divisiones (forks)* la información de un archivo: los datos que propiamente constituyen al archivo en cuestión son la *división de datos (data fork)* y la información *acerca del archivo* se guardan en una estructura independiente llamada *división de recursos (resource fork)*.

  Esta idea resultó fundamental para presentar un entorno gráfico que respondiera ágilmente, sin tener que buscar los datos base de una aplicación dentro de un archivo de mucho mayor tamaño. La *división de recursos* cabe en pocos sectores de disco.

  La división de recursos puede contener todo tipo de información; los programas ejecutables son los que le dan un mayor uso, dado que incluyen desde los archivos gráficos (ícono a mostrar para el archivo, ubicación de la ventana a ser abierta, etc.) hasta aspectos funcionales, como la traducción de sus cadenas al lenguaje particular del sistema en que está instalado. Esta división permite gran flexibilidad, dado que no es necesario tener acceso al fuente del programa para crear traducciones y temas.

  La división de recursos incluye un campo llamado *creador*, que indica cuál programa fué el que generó al archivo. Si el usuario solicita ejecutar un archivo de datos, SO lanzaría al programa *creador*, indicándole que abra al archivo en cuestión.

  Las versiones actuales de **MacOS** ya no emplean esta técnica sino una llamada *appDirectory*. La técnica base es la misma.

4.5. Archivos especiales

Dado que los sistemas de archivos son estructuras de tan fácil manejo y comprensión, pueden ser aprovechados para presentar todo tipo de estructuras, no sólo archivos. La máxima Unix: **Todo es un archivo**, resulta un ejemplo natural.

En un sistema **Unix** estándar, un archivo puede pertenecer a las siguientes categorías:

- **Archivo estándar**: El tipo básico de archivo es, precisamente, el archivo. Representa directamente la información que lo conforma.
- **Objetos del sistema de archivos**: La información acerca del sistema de archivos se almacena también dentro de archivos, aunque su tratamiento es especial: el SO oculta al archivo *real*, y gestiona su información procesada para que lo emplee el usuario. Este tipo de archivos puede referirse a directorios o a ligas simbólicas (*soft links*).
- **Dispositivos**: Permiten el acceso directo a un dispositivo externo. Pueden ser *de bloques* o *de caracteres*. El contenido del archivo son dos números, el *mayor* y el *menor*; tradicionalmente, el mayor indica de qué clase de dispositivo se trata, y el menor, la instancia de quá se trata.

  Por ejemplo, tanto los discos duros como sus particiones llevan por número mayor el ocho; los números menores para los duros son 0, 16, 32, 48. Cada uno de los discos puede tener hasta 15 particiones: 1 a 15, 17 a 31, etc.
- **Comunicación entre procesos**: Los archivos pueden representar *pipes* y *sockets*. Ambos son mecanismos que permiten intercambiar información entre distintos procesos; los pipes son más sencillos de emplear, pero permite únicamente la comunicación unidireccional, en tanto que el socket permite comunicación bidireccional como si se tratara de una conexión por red.

4.6. Transferencias orientadas a bloques

Un sistema de archivos es la representación que se da a un conjunto de archivos y directorios sobre un *dispositivo de bloques*; esto es un dispositivo que, para cualquier transferencia solicitada desde o hacia él, responderá con un bloque de tamaño predefinido.

Esto es, si bien el SO presenta una abstracción por medio de la cual la lectura (`read()`) puede ser de un tamaño arbitrario, todas las transferencias de datos desde cualquiera de los discos serán de un múltiplo del tamaño de bloques, definido por el hardware (típicamente 512 bytes).

Al leer, por ejemplo, solamente un registro de 75 bytes, el SO lee el bloque completo y probablemente lo mantiene en un caché en la memoria principal; si en vez de una lectura, la operación efectuada fué una de escritura (`write()`), y el sector a modificar no ha sido leído

aún a memoria (o fué leído hace mucho, y puede haber sido expirado del caché), el sistema tendrá que leerlo nuevamente modificarlo en memoria, y volver a guardarlo a disco.

5. Organización de archivos (Cap. 39)

Es común que en un sólo medio de almacenamiento de un equipo de uso casero aloje *decenas de miles* de archivos, por lo tanto se tiene que ver también cómo organizar una gran cantidad de archivos.

Directorio	**Contenido y Propósito**
`/`	Raíz del sistema de archivos. Todos los archivos y directorios cuelgan de aquí.
`/bin`	Binarios esenciales del sistema, accesibles a todos los usuarios. Ejemplos: `ls`, `cp`, `mkdir`.
`/boot`	Archivos necesarios para el arranque del sistema, como el kernel (`vmlinuz`) y GRUB.
`/dev`	Archivos especiales que representan dispositivos del sistema, como discos (`sda`) o terminales (`tty`).
`/etc`	Archivos de configuración del sistema y servicios. Ejemplos: `fstab`, `passwd`.
`/home`	Directorios personales de los usuarios. Ejemplo: `/home/usuario`.
`/lib`	Bibliotecas compartidas esenciales para los binarios en `/bin` y `/sbin`.
`/media`	Punto de montaje automático para dispositivos extraíbles como USB o discos externos.
`/mnt`	Punto de montaje manual para sistemas de archivos temporales.
`/opt`	Software adicional o de terceros instalado manualmente fuera del sistema base.
`/proc`	Sistema de archivos virtual con información del kernel y procesos en tiempo real.
`/root`	Directorio personal del superusuario (`root`).
`/run`	Información del sistema y procesos desde el último arranque, almacenada en RAM.
`/sbin`	Binarios esenciales para tareas administrativas, normalmente usados solo por el superusuario.
`/srv`	Datos de servicios proporcionados por el sistema, como servidores web o FTP.
`/sys`	Sistema de archivos virtual para interactuar con el kernel y los dispositivos.
`/tmp`	Archivos temporales. Generalmente se eliminan al reiniciar el sistema.
`/usr`	Archivos de usuario: aplicaciones, bibliotecas, documentación.
`/usr/bin`	Binarios de usuario no esenciales. Ejemplos: `python`, `gcc`, `firefox`.
`/usr/lib`	Bibliotecas compartidas para programas en `/usr/bin`.
`/usr/share`	Archivos compartidos, como documentación, íconos, etc.
`/var`	Archivos variables como logs, correo, spool de impresión, bases de datos temporales.

Cuadro 2: Directorios principales del sistema de archivos Unix y su función

5.1. El directorio como un *grafo dirigido*

El concepto dominante en almacenamiento hoy en día es el de *directorios jerárquicos*.

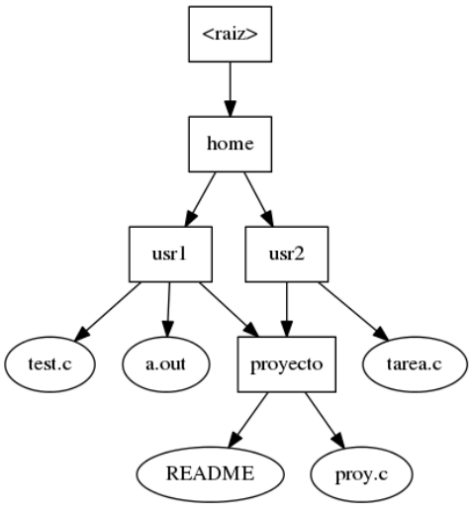


Figura 13: Directorio como un *grafo dirigido acíclico*: el directorio `proyecto` está tanto en el directorio `/home/usr1` como en el directorio `/home/usr2`

En un grafo dirigido como el presentado en la Figura 13, un mismo nodo puede tener varios directorios *padre*, permitiendo, por ejemplo, que un directorio de trabajo común sea parte del directorio personal de dos usuarios. Esto es, *el mismo objeto* está presente en más de un punto del árbol.

Un sistema de archivos puede permitir la organización como un *grafo dirigido*, aunque es común que la interfaz que presenta al usuario se restrinja a un *grafo dirigido acíclico*: las ligas múltiples son permitidas, siempre y cuando no generen un ciclo.

La semántica de los sistemas **Unix** implementa directorios como grafo dirigidos por medio de dos mecanismos:

- **Liga o enlace duro** (*Hard link*): La entrada de un archivo en un directorio Unix es la relación entre la ruta del archivo y el *número de i-nodo* en el sistema de archivos. Si a partir de un archivo en cualquier punto del árbol se crea un *hard link* a él, esta es sencillamente otra entrada en el directorio apuntando al mismo *i-nodo*. Ambas entradas, pues, son el mismo archivo - no hay uno *maestro* y uno *dependiente*.

  En un sistema Unix, este mecanismo tiene sólo dos restricciones:

  1. Sólo se pueden hacer hard links dentro del mismo volumen.
  2. No pueden hacerse hard links a directorios, sólo a archivos. [7]

- **Liga o enlace simbólico**(*soft link*): Es un archivo *especial*, que meramente indica a dónde apunta, o sea, contiene el path del archivo original. El encargado de seguir este

[7] Formalmente, puede haberlas, pero sólo el administrador puede crearlas.

archivo a su destino (esto es, *resolver* el soft link) es el SO mismo; un proceso no tiene que hacer nada especial para seguir el link.

Un soft link puede *apuntar* a directorios, incluso creando ciclos, o a archivos en otros volúmenes.

Cuando se crea un soft link, el link y el archivo son dos entidades distintas. Si bien cualquier proceso que abra al archivo destino estará trabajando con la misma entidad, en caso de que éste sea renombrado o eliminado, el link quedará *roto* (esto es, apuntará a una ubicación inexistente).

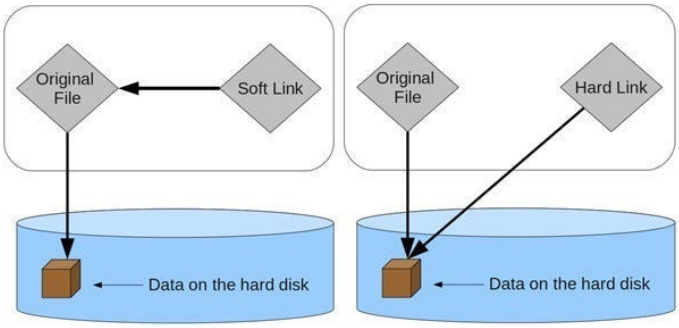


Figura 14: Hard link y soft link.

Si bien estos dos tipos de links son válidos también en los sistemas Windows [8], en dichos sistemas sigue siendo más común emplear los *accesos directos*. Se denomina así a un archivo (identificado por su extensión, *.lnk*), principalmente creado para poder *apuntar* a los archivos desde el escritorio y los menús; **si un proceso solicita al sistema abrir el *acceso directo*, no obtendrá al archivo destino, sino al acceso directo mismo**.

Ya con el conocimiento de los links, y reelaborando la Figura 13 con mayor apego a la realidad: en los SO (tanto Unix como Windows), todo directorio tiene dos entradas especiales: los directorios `.` y `..`, que aparecen tan pronto como el directorio es creado, y resultan fundamentales para entender la *navegabilidad* del árbol:

Como se puede ver en la Figura 15, en todos los directorios, `.` es un hard link al mismo directorio, y `..` es un link al directorio *padre* (de nivel jerárquico inmediatamente superior). Como sólo puede haber un link `..`, un directorio enlazado desde dos lugares distintos sólo puede apuntar hacia uno de ellos con su link `..`; en este caso, el directorio común `proyecto` está dentro del directorio `/home/usr2`. La figura representa el *soft link* desde `/home/usr1` como una línea punteada.

Hay una excepción a esta regla: el directorio raíz (*root*). En este caso, tanto `.` como `..` apuntan al mismo directorio.

[8] Únicamente en aquellos que emplean el sistema de archivos que introdujo Windows NT, el NTFS, no en los que utilizan alguna de las variantes FAT.

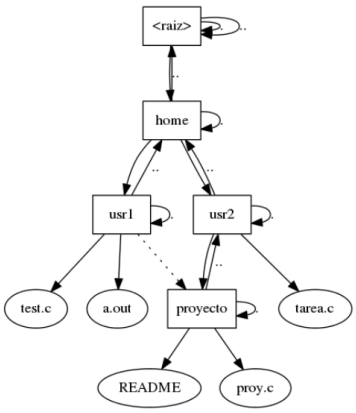


Figura 15: Directorio como un *grafo dirigido*, mostrando los *links ocultos* al directorio actual **.** y al directorio *padre* **..** .

Esta es la razón por la cual no se puede tomar rigurosamente un árbol de archivos como un *grafo dirigido acíclico*, ni en Windows ni en Unix: tanto las entradas **.** (al apuntar al mismo directorio donde están contenidas) como las entradas **..** (al apuntar al directorio padre) crean ciclos.

5.2. Operaciones con directorios

Al igual que los archivos, los directorios tienen una semántica básica de acceso. Éstos resultan también ser tipos de datos abstractos con algunas operaciones definidas:

- **Abrir y cerrar**: Un programa que requiera trabajar con un directorio deberá *abrirlo* para hacerlo y *cerrarlo* cuando ya no lo requiera. En C, se usan las funciones *opendir()* y *closedir()*. Estas funciones trabajan asociadas a un *flujo de directorio* (*directory stream*), que funciona de forma análoga a un file descriptor.

- **Listado de archivos**: Para mostrar los archivos que conforman un directorio, una vez que éste se *abre*, el programa *lee* (con *readdir()*) cada una de sus entradas. Cada uno de los resultados es una estructura *dirent* (*directory entry*) que contiene su nombre en *d_name*, un apuntador a su *i-nodo* en *d_ino*, y algunos datos adicionales del archivo en cuestión.

  Para presentar al usuario la lista de archivos que conforman un directorio, podría hacerse:

```
1         #include <stdio.h>
2         #include <dirent.h>
3         #include <sys/types.h>
4
5         int main(int argc, char *argv[]) {
6             struct dirent *archivo;
```

```
7               DIR *dir;
8               if (argc != 2) {
9                   printf("Indique el directorio a mostrar\n");
10                  return 1;
11              }
12              dir = opendir(argv[1]);
13              while ((archivo = readdir(dir)) != 0) {
14                  printf("%s\t", archivo->d_name);
15              }
16              printf("\n");
17              closedir(dir);
18          }
```

Este programa podría ser un *ls* tomando como argumento el directorio actual `.` (*ls .*).

Al igual que en archivos, se puede *reposicionar* (*seek()*) al file descriptor. Para *rebobinar* el descriptor del directorio al principio del listado se emplea *rewinddir()*.

- **Buscar un elemento**: La llamada *fopen()* antes descrita efectúa una búsqueda similar a la presentada en el ejemplo de código anterior, claro está, deteniéndose cuando encuentra al archivo en cuestión.
- **Crear, eliminar o renombrar un elemento**: Si bien estas operaciones se llevan a cabo sobre el directorio, son invocadas por medio de la semántica orientada a archivos: un archivo es creado con *fopen()* eliminado con *remove()*, y renombrado con *rename()*.

5.3. Recorriendo los directorios

Procesar todas las entradas de un directorio, incluyendo las de sus subdirectorios, se denomina *recorrer el directorio* (*directory traversal*).

Al considerar directorios como grafos dirigidos, se vuelve indispensable mantener en memoria la información de todos los nodos que ya han sido tocados - en caso contrario, al encontrarse un ciclo (incluso si este es creado por mecanismos de *soft links*), se corre el peligro de entrar en un bucle infinito.

Para recorrer el directorio ilustrado como ejemplo en la Figura 16, no bastaría tomar nota de las rutas de los archivos conforme son recorridas - cada vez que los sean procesados, su ruta será distinta. Al intentar respaldar el directorio `/home/jose/proyecto/`, por ejemplo, el recorrido resultante podría ser:

- `/home/jose/proyecto`
- `/home/jose/proyecto/miembros`
- `/home/jose/proyecto/miembros/jose`

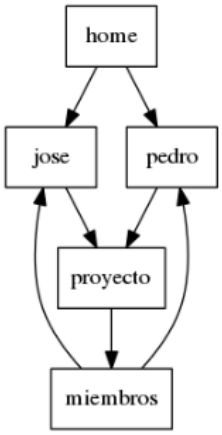


Figura 16: Directorio basado en grafo dirigido que incluye ciclos.

- `/home/jose/proyecto/miembros/jose/proyectos`
- `/home/jose/proyecto/miembros/jose/proyectos/miembros`
- ... y un etcétera infinito.

Para resolver esta situación, los programas que recorren directorios en los sistemas de archivos *reales*, deben *emplear un indexado basado en i-nodo*, que identifica sin ambigüedad a cada uno de los archivos.

5.4. *Montaje* de directorios

Para trabajar con el contenido de un sistema de archivos, el SO tiene que *montarlo*: ubicarlo en algún punto del árbol de archivos visible al sistema y al usuario.

Es muy común que un SO trabaje con distintos sistemas de archivos al mismo tiempo. Esto puede obedecer a varias causas, entre las cuales se encuentran:

- **Distintos medios físicos**: Si la PC tiene más de una unidad de almacenamiento, el espacio dentro de cada uno de los discos se maneja como un sistema de archivos independiente. Esto es especialmente cierto en la presencia de unidades removibles (`CD`, unidades `USB`, discos duros externos, etc...).
- **Diferentes usos esperados**: Distintos *esquemas de organización* (distintos sistemas de archivos), presentan ventajas para diversos patrones de uso.
- **Abstracciones de sistemas no-físicos**: El SO puede presentar diversas estructuras *con una estructura* de sistema de archivos. El ejemplo más claro de esto es el *sistema de archivos virtual* `/proc`, presente en los sistemas Unix, que permite ver diversos aspectos de los procesos en ejecución (y, en Linux, del sistema general). Los archivos bajo `/proc`

no están en ningún disco, `/proc` se genera *dinámicamente en memoria*, sin embargo, su contenido se presenta como si fueran archivos estándar.

- **Razones administrativas**: El administrador del sistema puede emplear sistemas de archivos distintos para aislar espacios de usuarios entre sí: por ejemplo, para evitar que un exceso de mensajes enviados en la bitácora (típicamente bajo `/var/log`) saturen al sistema de archivos principal, o para determinar patrones de uso máximo por grupos de usuarios.

En los sistemas tipo Unix, el mecanismo para montar los archivos es el de un árbol con *puntos de montaje*. Esto es, *todos los archivos y directorios* del SO están estructurados en un *único árbol*. Cuando se solicita al SO *montar* un sistema de archivos en un determinado directorio, éste se integra al árbol, haciendo *inaccesible temporalmente* el contenido previo de dicho directorio hasta que se desmonte el nuevo sistema de archivos.

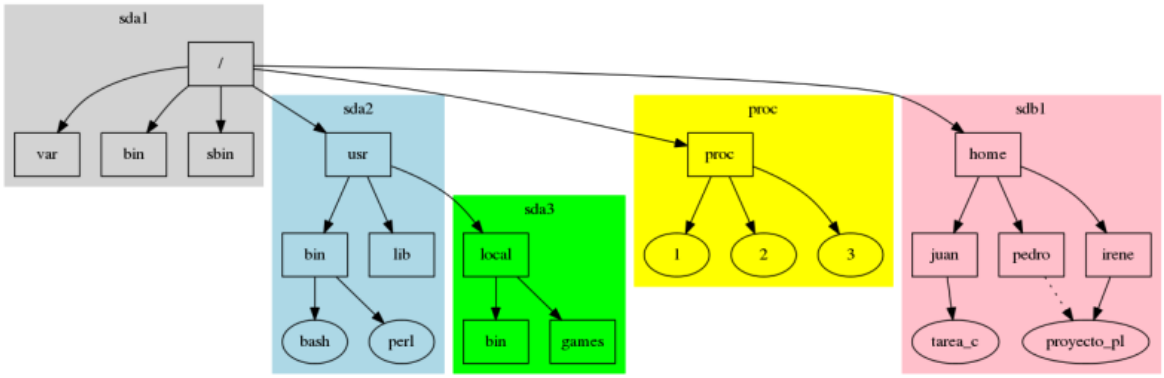


Figura 17: Árbol formado del montaje de `sda1` en la raíz (`/`), `sda2` como `/usr`, `sdb1` como `/home`, y el directorio virtual `proc`.

La manera en que esto se presenta en sistemas Windows es muy distinta. Ahí, cada uno de los volúmenes *detectados* recibe un *identificador de volumen*, y es montado automáticamente en un sistema de directorio estructurado como árbol de un solo nivel representando a los dispositivos del sistema [9] Este árbol es presentado a través de la interfaz gráfica como *Mi PC* (aunque este nombre no significa nada para la `API` del sistema).

Para especificar la *ruta completa* a determinado archivo, se inicia con el identificador de volumen. De este modo, la especificación absoluta de un archivo es una cadena como `VOL:\Dir1\Dir2\Archivo.ext` - el carácter `:` separa al volumen del árbol del sistema de archivos, y el carácter `\` separa uno de otro a los directorios. Si no se especifica la unidad, el sistema asumirá que se está haciendo referencia a la *unidad actual* (la última unidad en ser utilizada).

[9] En realidad, este árbol no sólo incluye a los volúmenes de almacenamiento, sino que a los demás dispositivos del sistema, como los distintos puertos, pero los *oculta* de la interfaz gráfica.

Los volúmenes `A` `B` están reservados para las unidades de disco flexible; `C` se refiere al disco duro de arranque, y las unidades posteriores que va detectando el sistema son `D`, `E`, `F`, etcétera...

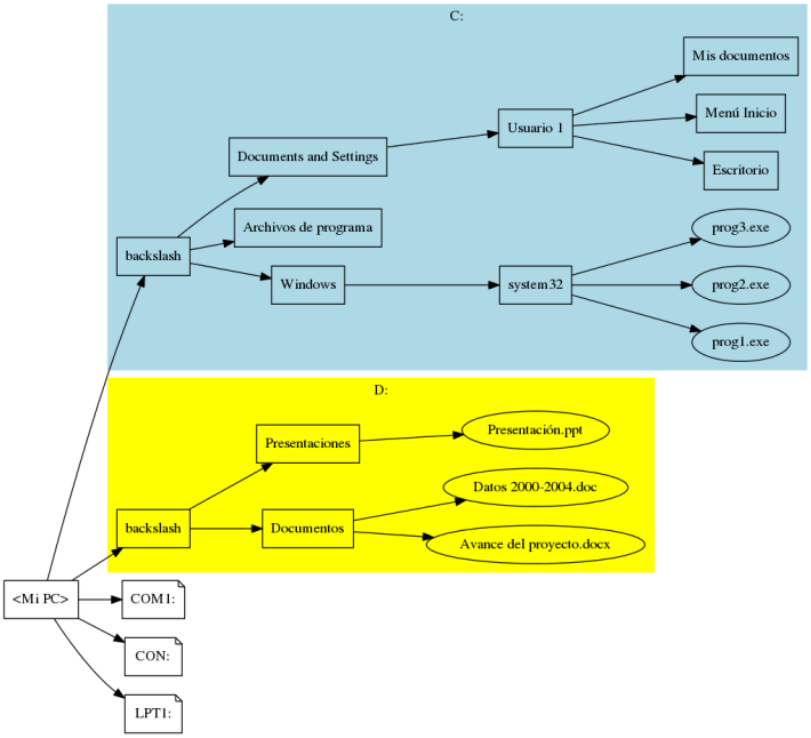


Figura 18: Jerarquía de dispositivos y volúmenes de archivos en un sistema Windows.

En resumen, en Unix se montan los sistemas de archivos en un directorio, ocultando todo lo que dicho directorio tiene. En Windows, el sistema de archivos se monta automáticamente en *Mi PC*, no ocultando nada.

6. Control de acceso (Cap. 40)

Una parte importante de la información que se almacena respecto a cada uno de los archivos, directorios, u otros objetos son sus *permisos de acceso*. Éstos indican al sistema qué puede hacerse y qué no con cada uno de los objetos, posiblemente diferenciando el acceso dependiendo del usuario o clase de usuarios de que se trate. Veremos tres esquemas de control de acceso.

6.1. Sistemas FAT

El sistema de archivos FAT [10] fue diseñado a fines de los 70. Su diseño es suficientemente simple para ser incluido en dispositivos muy limitados. Con sólo modificaciones relativamente

[10] Recibe su nombre de las siglas de su estructura fundamental, la *Tabla de Asignación de Archivos*, o *File Allocation Table*.

menores a su diseño original, ha logrado extenderse de un sistema pensado para volúmenes de 150KB hasta llegar a las decenas de gigabytes.

En cada una de las entradas del directorio en un sistema FAT, el *byte número 12* almacena los siguientes *atributos*:

- **Oculto**: Si está encendido, el archivo no se deberá mostrar al usuario en listados de directorio.
- **Sólo lectura**: El SO debe evitar toda modificación al archivo, esto es, rechazar todas las llamadas al sistema que soliciten esta modificación.
- **Sistema**: Un sistema de archivos FAT tiende conforme se va utilizando a *fragmentar* los archivos que almacena. La carga del SO tiene que realizarse empleando código lo más compacto y sencillo posible: sin siquiera tener conocimiento del sistema de archivos. Este atributo indica que el SO debe cuidar no mover ni fragmentar al archivo en cuestión.
- **Archivado**: En MS-DOS, conforme se realizaba un respaldo, todos los archivos iban siendo marcados como *archivados*. Cuando un archivo era modificado, el SO retiraba este atributo. De esta manera, el siguiente respaldo contendría únicamente los archivos que habían sido modificados desde el respaldo anterior.

Vemos que para representar estos cuatro atributos basta con utilizar 4 bits. Dado que estamos reservando un byte entero para esta sección, algunos sistemas extendieron los atributos cubriendo distintas necesidades. Sin embargo, estos cuatro atributos son la base del sistema.

En MS-DOS no se hace diferencia entre *lectura* y *ejecución*. Este sistema basa la identificación de archivos en su *extensión*. Siendo un sistema concebido como *monousuario*, no tiene sentido condicionar la *lectura* de un archivo.

6.2. Modelo tradicional Unix

Unix está concebido para sistemas multiusuarios, por lo que tiene provisiones necesarias para que se indique qué usuarios tienen diferentes tipos de acceso a los archivos.

En Unix, cada uno de los usuarios pertenece a uno o más *grupos*. Estos grupos son gestionados por el administrador del sistema. Cada objeto en el sistema de archivos describe sus permisos de acceso por **nueve bits**, y con el identificador de usuario y grupo propietarios.

El primer bit indica qué clase de archivo es:

- **-**: Archivo normal.
- **d**: Directorio.
- **l**: Soft link.

Los siguientes bits están dispuestos en tres grupos de tres; uno para el *usuario*, uno para el *grupo* y uno para los *otros*. Cada uno de estos tres grupos indica si se otorga permiso de *lectura*, *escritura* o *ejecución*.

Estos tres grupos de tres permiten suficiente *granularidad* en el control de acceso para expresar casi cualquier combinación requerida.

Por ejemplo, supongamos que los usuarios `jose`, `pedro` y `teresa` están colaborando en el desarrollo de un sistema de facturación. Pueden solicitar al administrador del sistema la creación de un grupo, `factura`. Una vez creado el grupo, y suponiendo que el desarrollo se efectúa en el directorio `/usr/local/factura/`, podrían tener los siguientes archivos:

```
$ ls -la /usr/local/factura/
drwxrwxr-x   root   factura  4096   .
drwxr-xr-x   root   root     4096   ..
-rw-r-----   root   factura  12055  compilacion.log
-rwxrwxr-x   pedro  factura  114500 factura
-rw-rw-r--   teresa factura  23505  factura.c
-rw-rw-r--   jose   factura  1855   factura.h
-rw-rw----   teresa factura  36504  factura.o
-rw-rw----   teresa factura  40420  lineamientos.txt
-rw-rw-r--   pedro  factura  3505   Makefile
```

Este ejemplo muestra que cada archivo puede pertenecer a otro de los miembros del grupo. Los archivos pertenecen al usuario que los creó, a menos que su propiedad sea *cedida* por el administrador a otro usuario.

Para el ejemplo aquí mostrado, el usuario propietario es menos importante que el grupo al que pertenece.

Para los directorios, se emplea el mismo sistema de permisos; el permiso de escritura indica quién puede crear o eliminar archivos dentro de él, el de lectura indica qué usuarios pueden averiguar la lista de archivos contenidos en dicho directorio, y el de ejecución indica qué usuarios pueden *entrar* al directorio.

Claro está, el directorio `..` es del sistema, y permite sólo la lectura y ejecución a todos los usuarios.

En un sistema Unix, los dispositivos del sistema así como varios objetos virtuales (como los *pipes* de comunicación entre procesos) son representados en el sistema de archivos. Esto significa que, por medio de estos permisos, el administrador del sistema puede indicar qué usuarios pueden tener permisos de lectura o modificación sobre los diferentes componentes de hardware del sistema.

El esquema de permisos de Unix presenta algunas características adicionales, como los bits `SUID/SGID` o los *atributos extendidos* que presentan ciertas implementaciones.

Cuadro 3: Permisos de archivos en `/usr/local/factura/`

Nombre	**Tipo**	**Dueño**	**Grupo**	**Permisos**	**Significado**
`.`	dir	root	factura	drwxrwxr-x	Este directorio; grupo `factura` puede leer, escribir y entrar
`..`	dir	root	root	drwxr-xr-x	Directorio padre
`compilacion.log`	archivo	root	factura	-rw-r——	Solo `root` puede leer y escribir; grupo `factura` solo leer
`factura`	binario	pedro	factura	-rwxrwxr-x	Ejecutable por todos; editable por `pedro` y el grupo
`factura.c`	código	teresa	factura	-rw-rw-r–	Todos pueden leer; `teresa` y el grupo pueden escribir
`factura.h`	código	jose	factura	-rw-rw-r–	Igual que `factura.c`
`factura.o`	objeto	teresa	factura	-rw-rw—-	Solo `teresa` y el grupo pueden leer y escribir; otros sin acceso
`lineamientos.txt`	texto	teresa	factura	-rw-rw—-	Igual que `factura.o`
`Makefile`	texto	pedro	factura	-rw-rw-r–	Todos leen; `pedro` y el grupo escriben

Ya vimos que el comando `chmod` permite cambiar los bits de **modo** de cierto archivo. Otro comando útil es `chattr` el cual cambia los bits de **atributos** de un archivo. Estos se refieren a configuraciones propias del mismo.

6.3. Listas de control de acceso

Una de las desventajas del modelo tradicional de usuarios Unix es que requiere de la intervención del administrador del sistema para cada nuevo proyecto. Además, encontrar la representación justa para indicar un conjunto específico de permisos puede ser complicado. Dado que un archivo no puede pertenecer a más de un grupo, puede hacerse necesario crear conjuntos de grupos adicionales que intersectan aspectos específicos.

Varios SO, incluyendo particularmente las versiones de Windows derivadas del NT, han agregado *listas de control de acceso* (ACI - *Access Control Lists*): cada uno de los archivos tiene asociada una lista donde se indican los permisos particulares que tendrá sobre éste cada usuario o grupo particular.

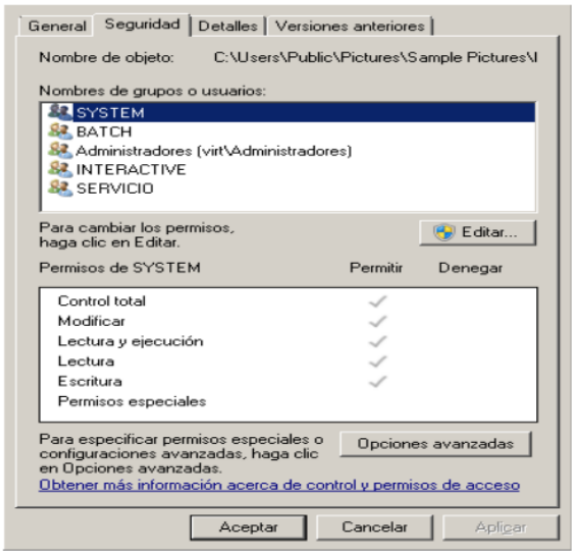


Figura 19: Lista de control de acceso a un archivo en Windows 7: en esta vista, se muestra que el grupo `SYSTEM` tiene permiso para todo tipo de acceso, con excepción del definido por *Permisos especiales*.

Si bien este modelo brinda un control mucho más fino, no está exento de desventajas:

1. Todos los permisos particulares que tenga un archivo tienen que almacenarse junto con su *i-nodo*. Dado que su longitud es variable, se presentan como una estructura adicional adjunta a los datos del archivo, y esto se traduce en un tiempo de acceso ligeramente mayor.

2. Resulta más difícil presentar un listado compacto y completo de los permisos para todos los archivos en determinado directorio. Realizar una evaluación *al vuelo* de los controles de acceso se vuelve más complejo para el administrador.

3. Da lugar a ambigüedades, cuya política de resolución debe establecerse *a priori*. Por ejemplo, si un usuario pertenece a dos grupos distintos, uno de ellos con *aprobación explícita* para la escritura a cierto archivo, y otro con *denegación explícita* para la escritura al mismo, debe haber una política global para resolver si el permiso será otorgado o rechazado. Estas políticas no siempre resultan fáciles de comprender por los usuarios del sistema.

   En Unix esto no pasa ya que el sistema no permite que un archivo pertenezca a más de un grupo.

7. Sistemas de archivos remotos (Cap. 41)

Es posible realizar transferencias de archivos entre computadoras de forma *implícita* y *automática*, esto es, sin necesidad de usar *explícitamente* un programa tipo `FTP`, lo cual sería

una *transferencia manual*.

Esto se puede lograr empleando *sistemas de archivos remotos*. Los mismos se presentan como un caso particular de la *abstracción de sistemas no-físicos*. Si bien el SO no tiene acceso *directo* a los archivos y directorios que le solicitará el usuario, por medio de los módulos de red, sabe cómo obtenerlos y presentarlos *cómo si fueran locales*.

Cuando se habla de sistemas de archivos en red, casi siempre se habla de un *modelo cliente-servidor*. Este término se refiere al papel que desempeña la computadora *dentro de cada conexión*, esto es, se designa como *cliente* a la computadora que solicita un servicio, y como *servidor* a la que lo provee; es frecuente que dos computadoras sean tanto servidor como cliente la una de la otra en distintos servicios.

7.0.1. Network File System (NFS)

El *Sistema de Archivos en Red* (*Network File System* - Sun Microsystems - 1984) está construido sobre el mecanismo `RPC` (*remote procedure call*), un mecanismo de mensajes y manejo básico de sesión que actúa como una capa superior a `TCP/IP` [11], incluyendo facilidades de *descubrimiento de recursos y abstracción*[12].

Estos mecanismos permiten al programador delegar en un *servicio* el manejo de las conexiones de red, particularmente la persistencia de sesiones en caso de desconexión, y limitar su atención a una *conexión virtual establecida*.

La motivación de origen para la creación de `NFS` fue presentar una solución que aprovechara el hardware ya común de dicha época, y que centralizara la administración: ofrecer las facilidades para contar con redes dónde hubiera un *servidor de archivos*, y dónde las estaciones de trabajo tuvieran únicamente una instalación básica [13], y el entorno de usuario completo estuviera disponible en cualquiera de las estaciones.

El sistema de archivos que ofrece sobre la red `NFS` cumple con la semántica Unix completa - para emplear un sistema remoto, basta montarlo [14] y usarlo como si fuera local. El manejo de permisos, usuarios e incluso los hard y soft links se manejan exactamente como se haría localmente.

El `NFS` viaja sobre un protocolo muy ligero; no implementa cifrado ni verificaciones adi-

[11] `TCP/IP` es el conjunto de protocolos que permite la comunicación entre dispositivos de una red. Este sistema de archivos se basa en `TCP/IP` para su funcionamiento pero opera a un nivel de abstracción más alto. Es como una API para `TCP/IP`.

[12] proceso de identificar y localizar dispositivos, servicios y otros recursos dentro de una red. Esto permite a los administradores de sistemas mapear la infraestructura de red, administrar el inventario de dispositivos y garantizar el acceso y la seguridad adecuados.

[13] Incluso manejando estaciones de trabajo *diskless*, esto es, computadoras sin disco duro, cuyo sistema de arranque tiene la capacidad de solicitar al servidor que le envíe incluso el núcleo del SO que ejecutará.

[14] Para montar un sistema remoto, se emplea un comando como `mount archivos.unr.ar:/ext/home /home`, con lo cual el directorio `/ext/home` ubicado en el servidor `archivos.unr.ar` aparecerá montado como directorio `/home` local.

cionales. Al día de hoy (creación libro Esteban), es uno de los mejores mecanismos para el envío de grandes cantidades de información, pero siempre en redes que sean *completamente confiables*.

Dado que se espera que la información de usuarios y permisos sea *consistente* en todos los clientes; Sun ofrecía también un esquema llamado *Yellow Pages* (posteriormente renombrado a `NIS`, *Network Information System*) para compartir la información de autenticación y listas de usuarios.

Las desventajas, en entornos sin `NIS`, es que los permisos se manejan según el identificador numérico del usuario. Si en diferentes sistemas el mismo usuario tiene distintos identificadores, los permisos no coincidirán. Es más, dado que el control de acceso principal es únicamente por dirección `IP`, para tener acceso irrestricto a los archivos de otros usuarios en `NFS` basta con tener control pleno de una computadora cualquiera en la red para poder *asumir o usurpar la identidad* de cualquier otro usuario.

Por último, para garantizar que las escrituras a archivos se llevaran a cabo cuando eran solicitadas (en contraposición de asumir éxito y continuar), todas las escrituras en un principio sobre `NFS` eran manejadas de forma *síncrona*, esto es, tras grabar un archivo, el cliente no continuaba con la ejecución hasta no tener confirmación por parte del servidor de que los datos estaban ya guardados en disco. Esto, si bien asegura que el usuario recibirá retroalimentación confiable respecto a la operación que realizó, ocasiona demoras que muchas veces son percibidas como excesivas. `NFS` es aún un sistema de archivos en red muy ampliamente empleado (de nuevo, al momento de la escritura del libro de Esteban).

7.1. Common Intenet File System (CIFS)

Es el equivalente a NFS en los entornos donde predominan los sistemas Windows. Aparece en los primarios de Microsoft alrededor de 1990.

Las primeras implementaciones estaban basadas en el protocolo NBF, frecuentemente conocido como NetBEUI, un protocolo no ruteable diseñado para redes pequeñas y entornos sencillos de oficina. A partir de Windows 2000 se ha reimplementado completamente para operar sobre `TCP/IP`. A partir de ese momento se lo denomina CIFS ya que había sido creado con el nombre de SMB (*server message block*). Sin embargo, este último nombre sigue siendo ampliamente utilizado.

El sistema CIFS se ajusta mucho más a la semántica de los sistemas frecuentemente encontrados en las PC, MSDOS y Windows, aunque dado el lapso transcurrido desde su introducción y el volumen de cambios en su mercado objetivo, ha pasado por varios cambios fundamentales que al día de hoy complican su uso.

Para tener acceso a un volumen compartido por SMB se introdujo el comando `NET`, basta indicar desde la línea de comando a DOS o Windows `NET USE W: \\servidor \directorio` para que el recurso compartido bajo el nombre `directorio` dentro del equipo conocido como

`servidor` aparezca en el árbol *Mi PC*, y el usuario pueda emplear sus contenidos como si fuera un sistema de archivos local, con un volumen asignado en `W:`.

Cuando LAN Manager fué introducido al mercado, los sistemas Microsoft no manejaban aún el concepto de usuarios, por lo que la única medida de seguridad que implementaba SMB era el manejo de hasta dos contraseñas por directorio compartido: con una, el usuario obtenía el acceso de sólo lectura, y con la otra, de lectura y escritura. Tras la aparición de Windows NT, se agregó un esquema de identificación por usuario/contraseña, que posibilita el otorgamiento de permisos con una *granularidad* mucho menor. [15]

El protocolo empleado por SMB fué pensado originalmente para una red *pequeña*, con hasta un par de decenas de equipos. La mayor parte de los paquetes eran enviados en modo de *difusión (broadcast)*, por lo que era fácil llegar a la saturación, y no había un esquema centralizado de resolución de nombres, por lo que era frecuente *no encontrar* a determinado equipo.

Los cambios que CIFS presenta a lo largo de los años son muy profundos. Las primeras implementaciones presentan fuertes problemas de confiabilidad, rendimiento y seguridad, además de estar planteadas para su uso en un sólo tipo de SO; al día de hoy, todos esos puntos han mejorado fuertemente. En sistemas Unix, la principal implementación, *Samba*, fue creada haciendo ingeniería inversa sobre el protocolo; a lo largo de los años, se ha convertido en un esquema tan robusto que es hoy por hoy tomado como implementación de referencia.

7.2. Andrew File System (AFS)

Ambos modelos mencionados hasta ahora comparten una visión *tradicional* del modelo cliente-servidor: al ver el comando que inicializa una conexión, e incluso al ver la información que guarda el núcleo del cliente respecto a cualquiera de los archivos, resulta claro cuál es el servidor para cada uno de ellos.

Andrew File System (1989), o AFS, plantea presentar un verdadero *sistema de archivos distribuido*, en el cual los *recursos compartidos* no tengan que estar en un servidor en particular, sino que un conjunto de equipos se repartan la carga (esto es, *agnosticismo a la ubicación*). AFS busca también una *fácil escalabilidad*, la capacidad de agregar tanto espacio de almacenamiento como equipos con papeles de servidor. AFS permite inclusive migrar completamente un volumen mientras está siendo empleado de forma transparente.

Ante la complejidad e inestabilidad adicional que presentan con tanta frecuencia las redes grandes [16]: AFS debe operar tan confiablemente como sea posible, *incluso sin la certeza de que la red opera correctamente.*

La autenticación en AFS se basa fuertemente sobre el modelo de *tickets* y credenciales de

[15] Esto significa que puede controlarse el acceso permitido más finamente, tanto por archivo como por usuario individual.

[16] El uso típico de AFS se planteaba para organizaciones grandes, del orden de decenas de miles de estaciones.

Kerberos [17], pero se aleja sensiblemente de la semántica de operación de archivos que hasta ahora se han presentado. Muchos eventos, operaciones y estados van ligados al *momento en el tiempo* en que se presentan, mediante un *modelo de consistencia débil* (*weak consistency model*). Muy a grandes rasgos, esto significa que:

- Cuando se abre un archivo, éste se copia completo al cliente. Todas las lecturas y *escrituras* (a diferencia de los esquemas tradicionales, en que éstas son enviadas al servidor lo *antes posible* y de forma síncrona) se dirigen únicamente a la copia local.
- Al cerrar el archivo, éste se copia de vuelta al *servidor de origen*, el cual se *compromete* a notificar a los clientes si un archivo abierto fué modificado (esto es, a *hacer una llamada o callback*). Los clientes pueden entonces intentar incorporar los cambios a su versión de trabajo, o continuar con la copia ya obtenida - es *de esperarse* que si un segundo cliente realiza alguna modificación, incorpore los cambios hechos por el primero, pero esta responsabilidad **se deja a la implementación del programa en cuestión**.

Esto significa, en pocas palabras, que los cambios a un archivo abierto por un usuario no son visibles a los demás de inmediato; sólo una vez que se cierra un archivo, los cambios hechos a éste son puestos a disposición de las sesiones abiertas actuales, y sólo son enviados como *versión actual* a las sesiones abiertas posteriormente.

AFS no busca ser un sistema para todo uso ni un reemplazo universal de los sistemas de archivos locales, en contraposición de los sistemas de archivos centralizados. AFS no plantea en ningún momento una operación *diskless*. Bajo el esquema aquí descrito, las lecturas y escrituras resultan baratas, porque se realizan exclusivamente sobre el caché local, pero abrir y cerrar un archivo puede ser muy caro, porque debe transferirse el archivo completo.

Si una base de datos se distribuyera sobre AFS, la carencia de mecanismos de bloqueo sobre *secciones* del archivo, y el requisito de operar sobre *archivos completos* harían impracticable compartir un archivo de uso intensivo y aleatorio.

8. Plasmando la estructura en el dispositivo (Cap. 43)

8.1. La eliminación de entradas del directorio (43.4.3)

Sólo unos pocos sistemas de archivos guardan sus directorios ordenados - si bien esto facilitaría las operaciones más frecuentes que se realizan sobre ellos (en particular, la búsqueda), mantenerlo ordenado ante cualquier modificación resultaría mucho más *caro*, dado que tendría que reescribirse el directorio completo al crearse o eliminarse un archivo dentro de éste. y lo que es más importante, más *peligroso*, dado que aumentaría el tiempo que los datos del directorio están en un estado inconsistente, aumentando la probabilidad de que ante una

[17] Un sistema de autenticación y autorización centralizado para entornos corporativos.

interrupción repentina (fallo de sistema, corte eléctrico, desconexión del dispositivo, etc.) se presentara corrupción de la información, llevando a pérdida de datos. Al almacenar las entradas del directorio sin ordenar, las escrituras que modifican esta crítica estructura se mantienen atómicas: un sólo sector de 512 bytes puede almacenar 16 entradas básicas de FAT, de 32 bytes cada una [18].

Ordenar las entradas del directorio teniendo sus contenidos ya en memoria y, en general, diferir las modificaciones al directorio resulta mucho más conveniente en el caso general. Esto vale también para la eliminación de archivos - a continuación se abordará la estrategia que sigue FAT. Cabe recordar que FAT fue diseñado cuando el medio principal de almacenamiento era el disco flexible, decenas de veces más lento que el disco duro, y con mucha menor confiabilidad.

Cuando se le solicita a un sistema de archivos FAT eliminar un archivo, éste no se borra del directorio, ni su información se libera de la tabla de asignación de archivos, sino que se *marca* para ser ignorado, reemplazando el primer carácter de su nombre por `0xE5`. Ni la entrada de directorio, ni la *cadena de clusters* correspondiente en las tablas de asignación, son eliminadas - sólo son marcadas como *disponibles*. El espacio de almacenamiento que el archivo eliminado ocupa debe, entonces, ser *sumado* al espacio libre que tiene el volumen (en NachOS marcamos los sectores que ocupa el archivo como libres en el bitmap). Es sólo cuando se crea un nuevo archivo empleando esa misma entrada, o cuando otro archivo ocupa el espacio físico que ocupaba el que fue eliminado, que el SO marca *realmente* como desocupados los *clusters* en la tabla de asignación. Es por esto que desde los primeros días de las PC hay tantas herramientas de recuperación (o *des-borramiento*, *undeletion*) de archivos: siempre que no haya sido creado uno nuevo que ocupe la entrada de directorio en cuestión, recuperar un archivo es tan simple como volver a ponerle el primer carácter a su nombre.

Este es un ejemplo de un *mecanismo flojo* (en contraposición de los *mecanismos ansiosos*, vistos en la sección 34.1). Eliminar un archivo requiere de un trabajo mínimo, mismo que es *diferido* al momento de reutilización de la entrada de directorio.

8.2. Compresión y *desduplicación*

Los archivos almacenados en un área dada del disco tienden a presentar patrones comunes. Algunas situaciones ejemplo que llevarían a estos patrones comunes son:

- Dentro del directorio de trabajo de cada uno de los usuarios hay típicamente archivos creados con los mismos programas, compartiendo encabezado, estructura y ocasionalmente incluso parte importante de los datos.
- Dentro de los directorios de binarios del sistema, habrá muchos archivos ejecutables

[18] Aunque en el caso VFAT, las diferentes entradas que componen un sólo nombre de archivo pueden quedar separadas en diferentes sectores.

compartiendo el mismo *formato binario*.

- Es muy común también que un usuario almacene versiones distintas del mismo documento.
- Dentro de un mismo documento, es frecuente que el autor repita en numerosas ocasiones las palabras que describen sus conceptos principales.

Conforme las computadoras aumentaron su poder de cómputo, desde finales de los 80 se presentaron varios mecanismos que permitían aprovechar las regularidades en los datos almacenados en el disco para comprimir el espacio utilizable en un mismo medio. La compresión típicamente se hace por medio de mecanismos estimativos derivados del análisis del contenido, [19] que tienen como resultado un nivel variable de compresión: con tipos de contenido altamente regulares (como podría ser texto, código fuente, o audio e imágenes *en crudo*), un volumen puede almacenar frecuentemente mucho más del 200 % de su espacio real.

Con contenido poco predecible o con muy baja redundancia (como la mayor parte de formatos de imágenes y audio, que incluyen ya una fase de compresión, o empleando cualquier esquema de cifrado) la compresión no ayuda y puede reducir el rendimiento global del sistema en que es empleada.

8.3. Compresión de volumen completo

El primer sistema de archivos que se popularizó fue *Stacker*. Operaba bajo MS-DOS, creando un dispositivo de bloques virtual alojado en un disco estándar. [20]

Ahora, sumando la variabilidad derivada del enfoque probabilístico al uso del espacio con el ubicarse como una compresión orientada al volumen entero, resulta natural encontrar una de las dificultades resultantes del uso de estas herramientas: dado que el SO estructura las operaciones de lectura y escritura por bloques de dimensiones regulares (por ejemplo, el tamaño típico de sector de hardware de 512 bytes), al poder éstos traducirse a más o menos bloques reales tras pasar por una capa de compresión, es posible que el sistema de archivos tenga que reacomodar constantemente la información al *crecer* alguno de los bloques previos. Conforme mayor era el tiempo de uso de una unidad comprimida por *Stacker*, se notaba más degradación en su rendimiento.

Además, dado que bajo este esquema se empleaba el sistema de archivos estándar, las tablas de directorio y asignación de espacio resultaban también comprimidas. Estas tablas contienen la información fundamental del sistema de archivos; al comprimirlas y reescribirlas

[19] Uno de los algoritmos más frecuentemente utilizados y fáciles de entender es la *Codificación Huffman*; éste y la familia de algoritmos *Lempel-Ziv* sirven de base para prácticamente la totalidad de las implementaciones.

[20] Esto significa que, al solicitarle la creación de una unidad comprimida de 30MB dentro del volumen C (disco duro primario), ésta aparecería disponible como un volumen adicional. El nuevo volumen requería de la carga de un *controlador* especial para ser *montado* por el SO.

constantemente, la probabilidad de que resulten dañadas en alguna falla (eléctrica o lógica) aumenta. Si bien los discos comprimidos por *Stacker* y otras herramientas fueron populares, principalmente en la primera mitad de los noventa, conforme aumentó la capacidad de los discos duros fue declinando su utilización.

8.4. Compresión archivo por archivo

Para el entonces nuevo sistema de archivos NTFS, Microsoft decidió incluir una característica distinta, más segura y más modular: mantener el sistema de archivos funcionando de forma normal, sin compresión, y habilitar la compresión *archivo por archivo* de forma transparente al usuario.

Este esquema permite al administrador del sistema elegir, por archivos o carpetas, qué áreas del sistema de archivos desea almacenar comprimidas; esta característica viene como parte de todos los SO Windows a partir de la versión XP, liberada en 2003.

Si bien la compresión transparente por archivo se muestra mucho más atractiva que la de volumen completo, no es una panacea y es frecuente encontrar en foros en línea la recomendación de deshabilitarla. En primer término, comprimir un archivo implica que un cambio pequeño puede tener un efecto mucho mayor: modificar un bloque puede implicar que el tamaño final de los datos cambie, lo cual se traduciría a la reescritura del archivo desde ese punto en adelante; esto podría mitigarse insertando espacios para preservar el espacio ya ocupado, pero agrega complejidad al proceso (y abona en contra de la compresión). Los archivos comprimidos son además mucho más sensibles a la corrupción de datos, particularmente en casos de fallo de sistema o de energía: dado que un cambio menor puede resultar en la necesidad de reescribir el archivo completo, la ventana de tiempo para que se produzca un fallo se incrementa.

En archivos estructurados para permitir el acceso aleatorio, como podrían ser las tablas de bases de datos, la compresión implicaría que los registros no estarán ya alineados al tamaño que el programa gestor espera, lo cual acarreará necesariamente una penalización en el rendimiento y en la confiabilidad. [21]

Por otro lado, los formatos nativos en que se expresan los datos que típicamente ocupan más espacio en el almacenamiento de los usuarios finales implican ya un alto grado de compresión: los archivos de fotografías, audio o video se codifican bajo diversos esquemas de compresión aptos para sus particularidades. Comprimir un archivo que ya está comprimido

[21] Recordar que un procesador lee de a words (8 bytes en arquitecturas de 64 bits) no de a bytes. Si los datos están desalineados, quizás para leer una estructura que cabría perfectamente en una palabra tenga que leer dos. Muchas veces los procesadores agregan bytes de *padding* para alinear los datos dentro de alguna estructura (el acceso a datos desalineados quizás requiere lógica de movimientos de bytes y esto es tiempo perdido). Además, el bus de datos muchas veces transfiere datos en palabras alineadas (por ejemplo, bloques de 4 u 8 bytes alineados). Si se pide algo que no está alineado, se necesita lógica extra para desplazar, juntar bytes, etc.

no sólo no reporta ningún beneficio, sino que resulta en desperdicio de trabajo por el esfuerzo invertido en descomprimirlo cada vez que es empleado.

8.5. Desduplicación

Hay una estrategia fundamentalmente distinta para optimizar el uso del espacio de almacenamiento, logrando muy altos niveles de *sobreuso:* guardar *sólo una copia de cada cosa*.

Esto se conoce como **desduplicación**.

Que esta medida vuelva a adquirir popularidad se debe en buena parte a la *consolidación* de servidores ante la adopción a gran escala de mecanismos de virtualización. Dado que un mismo servidor puede estar alojando a decenas o centenas de *máquinas virtuales*, muchas de ellas con el mismo SO y programas base, los mismos archivos se repiten muchas veces; si el sistema de archivos puede determinar que cierto archivo o bloque ya está almacenado, podría guardar sólamente una copia.

La principal diferencia entre la desduplicación y los *hard links* es que, en caso de que cualquiera de estos archivos (o bloques) sea modificado, el sistema de archivos tomará el espacio necesario para representar estos cambios y evitará que esto afecte a los demás archivos. Además, si dos archivos inicialmente distintos se hacen iguales, se liberará el espacio empleado por uno de ellos de forma automática.

Para identificar qué datos están duplicados, el mecanismo más utilizado es calcular el *hash criptográfico* de los datos; este mecanismo permite una búsqueda rápida y confiable de coincidencias, ya sea por archivo o por bloque.

La desduplicación puede realizarse *en línea* o *fuera de línea* - esto es, analizar los datos buscando duplicidades al momento en que estos llegan al sistema, o (dado que es una tarea intensiva, tanto en uso de procesador como de entrada/salida) realizarla como una tarea posterior de mantenimiento, en un momento de menor ocupación del sistema.

Desduplicar por archivo es mucho más ligero para el sistema que hacerlo por bloque, aunque dadas las estructuras comunes que comparten muchos archivos, desduplicar por bloque lleva típicamente a una mucho mayor optimización del uso de espacio.

Al día de hoy, el principal sistema de archivos que implementa desduplicación es `ZFS`, desarrollado por Sun Microsystemas (hoy Oracle). En Linux, esta característica forma parte de `BTRFS`, aunque no ha alxandao los niveles de estabilidad como para recomendarse su uso en entornos de producción.

En Windows, esta funcionalidad se conoce como *single intance storage (almacenamiento de instancia única*.

El uso de desduplicación, particularmente cuando se efectúa por bloques, tiene un alto costo en memoria: para mantener buenos niveles de rendimiento, la tabla que relaciona el hash de datos con el sector en el cual está almacenado debe mantenerse en memoria. En el caso de la implementación de `ZFS` en FreeBSD, la documentación sugiere dedicar `5GB` de

memoria por cada TB de almacenamiento (0,5 % de la capacidad total).

8.6. Sistemas de archivos virtuales

Desde la década de los ochenta aparecieron diversas implementaciones de discos RAM: programas diseñados para tomar un espacio de memoria y darle la semántica de un disco, permitiendo manipular su contenido como si fuera un disco cualquiera. Esto es, presenta un almacenamiento volátil, pero de muy alta velocidad.

En los sistemas Linux, la norma es el uso de sistemas de archivos virtuales para el espacio de almacenamiento temporal /tmp, e incluso otros espacios tradicionalmente persistentes (como el directorio con los archivos de dispositivos /dev) se han convertido en virtuales.

8.6.1. Una ventana al núcleo

El directorio /proc en Linux posee montado un sistema de archivos virtual. Dicho sistema de archivos, en vez de representar un mapa de bloques en un dispositivo físico, presenta una ventana a la lista de procesos en ejecución en el sistema. /proc es hoy en día una suerte de ventana a estructuras internas del sistema: un árbol de directorios con una gran cantidad de archivos por medio de los cuales se puede monitorear el estado del sistema (memoria libre, número de procesos, consumo de procesador, etc.), e incluso modificar la configuración del sistema en ejecución.

Por ejemplo, en Linux, leer de /proc/sys/vm/swappiness dará, por omisión, un valor de 60. Escribir el valor 100 a este archivo tendrá por resultado inmediato que el sistema sea más *agresivo* con la paginación de memoria virtual, buscando dejar más espacio de memoria física disponible de lo que normalmente haría. Por otro lado, escribir el valor 1 tendrá por efecto que el SO realice la paginación sólo si no tiene otra alternativa. Un valor de 0 tendrá como efecto deshabilitar la paginación por completo. Este archivo, al igual que todos los que conforman /proc, se presenta ante los procesos en ejecución como un archivo de tipo normal; la particularidad radica en el sistema de archivos en que se ubica.

Claro está, por más que parezca una simple lectura de un archivo, el leer o modificar un valor dentro de /proc cruza también por una serie de llamadas al sistema; el que la configuración de la memoria virtual esté disponible mediante algo que *parece ser* un archivo (sin serlo) es precisamente un caso de abstracción hacia una interfaz consistente y conocida.

9. Fallos y recuperación (Cap. 45)

El sistema de archivos FAT es *relativamente frágil*: no es difícil que se presente una situación de *corrupción de metadatos*, y muy particularmente, que ésta afecte la estructura de las tablas de asignación. A lo largo de la vida de un sistema de archivos, conforme los archivos se van asignando y liberando, cambian su tamaño, y conforme el sistema de archivos se monta y

desmonta, pueden ir apareciendo *inconsistencias* en su estructura. En los sistemas tipo `FAT`, las principales inconsistencias son:

- **Archivos cruzados** (*cross-linked file*): La entrada en el directorio de un archivo incluye un apuntador al primer *cluster* de una *cadena*. Cada cadena debe ser única, esto es, ningún *cluster* debe pertenecer a más de un archivo. Si dos archivos incluyen al mismo *cluster*, esto indica una inconsistenciam y la única forma de resolverla es *truncar* uno de los archivos en el punto inmediato anterior a este cruce.

- **Cadenas perdidas o *huérfanas*** (*lost-clusters*): Cuando hay un espacio marcado como ocupado en la tabla de asignación, pero no hay ninguna entrada de directorio haciendo referencia a ella, el espacio está efectivamente bloqueado y, desde la perspectiva del usuario, inutilizado; además, estas cadenas pueden ser un archivo que el usario aún requiera.

  El directorio de Unix llamado `lost+found` ubicado en la raíz de todos los sistemas de archivos es usado por `fsck` (el equivalente en Unix de `CHKD SK`) para crear ligas a los archivos perdidos por corrupción de metadatos.

Cada sistema de archivos podrá presentar un distinto conjunto de inconsistencias, dependiendo de sus estructuras básicas y de la manera en que cada sistema operativo las maneja.

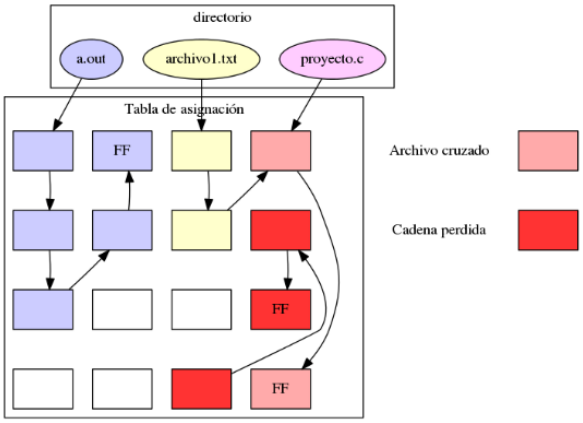


Figura 20: Inconsistencias en un sistema de archivos tipo `FAT`.

Los *controladores inteligentes* introdujeron muchos cambios que fueron disociando cada vez más al sistema operativo de la gestión física directa de los dispositivos. Si bien antes el SO podía determinar con toda certeza si una operación se había realizado o no, hoy en día los controladores dan un *acuse de recibo* a la información en el momento en que la colocan en el caché incorporado del dispositivo - en caso de un fallo de corriente, esta información puede no haber sido escrita por completo al disco.

Es importante que las operaciones con los metadatos que conforman el sistema de archivos sean atómicas. Por poner un ejemplo, crear un archivo en un volumen `FAT` requiere:

1. Encontrar una lista de *clusters* disponibles suficiente para almacenar la información que conformará al archivo.

2. Encontrar el siguiente espacio disponible en el directorio.

3. Marcar en la tabla de asignación la secuencia de *clusters* que ocupará el archivo.

4. Crear en el espacio encotrado una entrada con el nombre del archivo, apuntando al primero de sus *clusters.*

5. Almacenar los datos del archivo en cuestión en los *clusters* determinados en el paso 1.

Cualquier fallo que se presente después del tercer paso (tras efectuarse la primera modificación) tendrá como consecuencia que el archivo resulte corrupto, y muy probablemente, todo el sistema de archivos *presente inconsistencias* o *esté en un estado inconsistente.*

Si se intercambian los pasos 3 y 4 se puede dar lugar a un problema muy grande de corrupción. Como ejemplo, si se ejecuta primero 4 y luego se corta la energía, el espacio habrá sido creado y apuntado, pero no marcado en la tabla de asignación, por lo que cualquier otro proceso podría ocupar dicho espacio, pisando el archivo allí presente.

9.1. Datos y metadatos

En el ejemplo recién presentado, el sistema de archivos estará en un estado consistente siempre que se terminen los pasos 3 y 4 - la consistencia del sistema de archivos es independiente de la validez de los datos del archivo. Lo que busca el sistema de archivos, más que garantizar la integridad de los *datos* de uno de los archivos, es asegurar la de los *metadatos*: los datos que describen la estructura (de los datos en sí).

En caso de que un usuario desconecte una unidad a media operación, es muy probable que se presente pérdida de información, pero el sistema de archivos debe buscar no presentar ningún problema que ponga en riesgo *operaciones posteriores* o *archivos no relacionados.* La corrupción y recuperación de datos en archivos corruptos y truncados, cae más bien en el terreno de las aplicaciones de usuario.

9.2. Verificación de integridad

Cada SO incluye programas para realizar verificación (y en su caso, corrección) de la integridad de sus sistemas de archivos. En el caso de MS-DOS y Windows, estos programas son `CHKDSK` y `SCANDISK`; en los sistemas Unix, el programa general se llama `fsck`, y típicamente emplea a asistentes según el tipo de sistema que haya que revisar - `fsck.vfat, fsck.ext2`, etc.

Estos programas hacen un *barrido* del sistema de archivos, buscando evidencias de inconsistencia. Esto lo hacen, en líneas generales:

- Siguiendo todas las cadenas de *clusters* de archivos o tablas de i-nodos, y verificando que no haya archivos cruzados (compartiendo erróneamente bloques).
- Verificando que todas las cadenas de *clusters*, así como todos los directorios, sean alcanzables y sigan una estructura válida.
- Recalculando la correspondencia entre las estructuras encontradas y los diferentes bitmaps y totales de espacio vacío.

Estas operaciones frecuentemente duran entre decenas de minutos y horas, ya que son procesos intensivos y complejos. Además, deben ejecutarse teniendo acceso exclusivo al volumen a revisar, lo cual típicamente significa colocar al sistema completo en modo de mantenimiento.

Dado el elevado costo computacional, surgieron esquemas orientados a evitar la necesidad de invocar a estos programas de verificación: las *actualizaciones suaves*, los *sistemas de archivos con bitácora*, y los *sistemas de archivos estructurados en bitácora*.

9.3. Sistemas de archivo con bitácora

Tiene su origen en las bases de datos distribuidas. Consiste en separar un área del volumen y dedicarla a llevar una bitácora (*journal*) con todas las *transacciones* de metadatos. Una *transacción* es un conjunto de operaciones que deben aparecer como atómicas.

La bitácora se implementa generalmente como una *lista ligada circular*, con un apuntador que indica cuál fue la última operación realizada exitosamente. Periódicamente, o cuando la carga de transferencia de datos disminuye, el sistema verifica qué operaciones quedaron pendientes, y *avanza* sobre la bitácora, marcando cada una de las transacciones conforme las realiza.

En caso de tener que recuperarse de una condición de fallo, el SO sólo tiene que leer la bitácora, encontrar cuál fue la última operación efectuada y aplicar las restantes.

Una restricción de este esquema es que las transacciones guardadas en la bitácora deben ser *idempotentes* - esto es, si una de ellas es efectuada dos veces, el efecto debe ser exactamente el mismo que si hubiera sido efectuada una sola vez. Por poner un ejemplo, no sería válido que una transacción indicara "agregar al directorio x un archivo llamado y", dado que si la falla se produce después de procesar esta transacción pero antes de avanzar al apuntador de la bitácora, el directorio x quedaría con dos archivos y —una situación inconsistente. En todo caso, tendríamos que indicar "registrar al archivo y en la posición z del directorio x"; de esta manera, incluso si el archivo ya había sido registrado, puede volverse a hacerlo sin peligro.

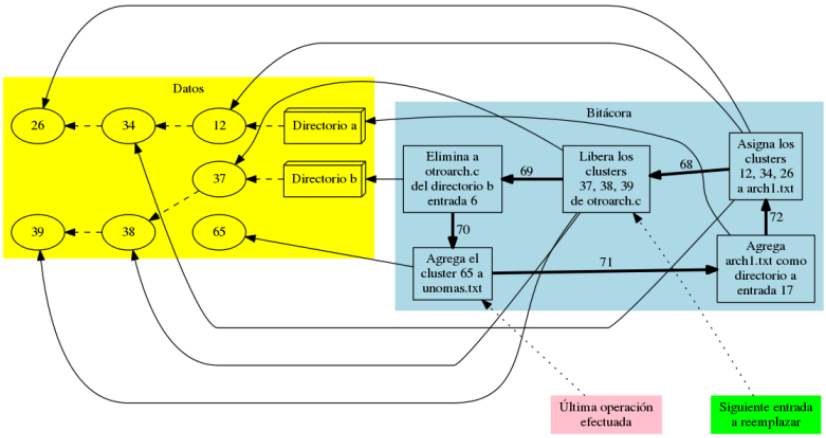


Figura 21: Sistema de archivos con bitácora.

Este esquema es el más utilizado hoy en día, y está presente en casi todos los sistemas de archivos modernos. Si bien con un sistema con bitácora no hace falta verificar el sistema de archivos completo tras una detención abrupta, esta no exime de que, de tiempo en tiempo, el sistema de archivos sea verificado: es altamente recomendable hacer una verificación periódica en caso de presentarse alguna corrupción, sea por algún bug en la implementación, fallos en el medio físico, o factores similarmente poco frecuentes.

La mayor parte de los sistemas de archivos incluyen contadores de *cantidad de montajes* y *fecha de último montaje*, que permiten que el SO determine, automáticamente, si corresponde hacer una verificación preventiva.

La bitácora es **guardada en el disco**.

En Unix, ejemplos de sistemas de archivos con bitácora son *ext3* y *ext4*. En Windows, *NTFS*.

9.4. Sistemas de archivos estructurados en bitácora

Si se lleva el concepto del sistema de archivos con bitácora a su límite, y se designa a *la totalidad* del espacio en el volumen como la bitácora, el resultado es un sistema de archivos *estructurado en bitácora* (*log-structured file systems*). Ahora no tengo que hacer dos escrituras, una en la bitácora y otra a disco. Hago una sola, ya que el disco es, a su vez, la bitácora.

Las ideas básicas detrás de la primera implementación de un sistema de archivos de esta naturaleza apuntan al empleo agresivo de caché de gran capacidad, y con un fuerte mecanismo de *recolección de basura*, reacomodando la información que esté más cerca de la *cola* de la bitácora (y liberando toda aquella que resulte redundante).

Este tipo de sistemas de archivos facilita las escrituras, haciéndolas siempre secuenciales, y busca, por medio del empleo del caché ya mencionado, evitar que las cabezas tengan que desplazarse con demasiada frecuencia para recuperar fragmentos de archivos.

Una consecuencia directa de esto es que los sistemas de archivos estructurados en bitáco-

ra fueron los primeros en ofrecer *fotografías* (*snapshots*) del sistema de archivos: es posible apuntar a un momento en el tiempo, y -con el sistema de archivos aún en operación- montar una copia de solo lectura con la información del sistema de archivos *completa* (incluyendo los datos de los archivos).

Estos sistemas de archivos, sin embargo, no están optimizados para cualquier carga de trabajo. Por ejemplo, una base de datos relacional, en la que cada uno de los registros es típicamente actualizado de forma independiente a los demás, y ocupa apenas fracciones de un bloque, resultaría tremendamente ineficiente.

Muchas de las implementaciones en sistemas libres han llegado a niveles de funcionalidad aceptables, pero por diversas causas han ido perdiendo el interés o el empuje de sus desarrolladores, y su ritmo de desarrollo ha decrecido.

Sin embargo, varios conceptos claves provenientes de este sistema, como los *snapshots*, se han ido aplicando a sistemas de archivos estándar.

Por otro lado, dado el fuerte crecimiento que están registrando los medios de almacenamiento de estado sólido, y dado que estos sistemas logran aprovechar mejor varias de sus características, es probable que el interés en esos sistemas de archivos resurja.